

Обзор автоматической настройки компилятора с использованием машинного обучения

АМИР Х. АШУРИ, Университет Торонто, Канада
 УИЛЬЯМ КИЛЛИАН, Пенсильванский университет в
 ДЖОН КАВАЗОС, Миллерсвилле, США Делавэрский университет, США
 ДЖАНЛУКА ПАЛЕРМО и КРИСТИНА СИЛЬВАНО, Politecnico di Milano, Italy

С середины 1990-х годов исследователи пытаются использовать подходы, основанные на машинном обучении, для решения ряда различных задач оптимизации компилятора. Эти методы в первую очередь повышают качество получаемых результатов и, что более важно, делают возможным решение двух основных задач оптимизации компилятора: выбор оптимизации (выбор того, какие оптимизации применять) и упорядочение по фазам (выбор порядка применения оптимизаций). Компилятор оптимизации пространства продолжает расти за счет продвижения АП-приложений, увеличивая количество компилятор оптимизаций и новых целевых архитектур. Общие этапы оптимизации в компиляторах не могут в полной мере использовать недавно введенные оптимизации и, следовательно, не могут идти в ногу с темпами увеличения возможностей. В этом обзоре обобщены и классифицированы последние достижения в использовании машинного обучения в области оптимизации компиляторов, в частности, по двум основным проблемам: (1) выбору наилучших оптимизаций и (2) поэтапному порядку оптимизаций. В обзоре освещаются подходы, применявшиеся до настоящего времени, полученные результаты, четкая классификация различных подходов и, наконец, влиятельные документы в этой области.

Концепции CCS: **Вычислительные методологии** → **Машинное обучение**; **Программное обеспечение и его разработка** →

Компиляторы;

Дополнительные ключевые слова и фразы: Автонастройка, упорядочение фаз, оптимизация, компиляторы, машинное обучение

Справочный формат ACM:

Амир Х. Ашури, Уильям Киллиан, Джон Кавазос, Джанлука Палермо и Кристина Сильвано. 2018. Обзор по автоматической настройке компилятора с использованием машинного обучения. *ACM Computat. Обзор*. 51, 5, статья 96 (сентябрь 2018 г.), 42 страницы.

<https://doi.org/10.1145/3197978>

1 Введение

Закон Мура [220] утверждает, что плотность транзисторов удваивается каждые два года. Однако темпы, с которыми улучшаются компиляторы, составляют порядка нескольких процентных пунктов в год. Компиляторы являются необходимыми инструментами, сокращающими разрыв между написанным программным обеспечением и целевым оборудованием. Существует множество нерешенных исследовательских задач в области компиляторов [115]. Вступление в эпоху после закона Мура [81], компиляторы изо всех сил стараются не отставать от растущих темпов развития постоянно расширяющегося аппаратного ландшафта (например, процессоров, графических процессоров, ПЛИС) и парадигм и моделей программирования программного обеспечения

Эта работа частично поддержана программой Комиссии ЕС H2020-FET-HPC в рамках гранта №. ANTAREX-671623.

Адреса авторов: А. Х. Ашури, Университет Торонто, Канада, 10 King's College Rd, Торонто, ОН М5S3G8; В. Киллиан, Миллерсвиллский университет Пенсильвании, США, 1 S George St, Миллерсвилл, Пенсильвания, 19551; Дж. Кавазос, Делавэрский университет, США, Ньюарк, DE, 19716; Г. Палермо и К. Сильвано, DEIB, Миланский политехнический институт, Италия, Via Ponzio 34/5, Милан 20133. Разрешение на создание цифровых или печатных копий всего или части этого произведения для личного использования или в классе предоставляется бесплатно при условии, что копии не создаются и не распространяются с целью получения прибыли или коммерческой выгоды и что копии содержат это уведомление и полную цитату на первой странице. Авторские права на компоненты этой работы, принадлежащие другим лицам, кроме ACM, должны соблюдаться. Допускается использование аннотаций с предоставлением кредита. Для копирования иным образом или переиздания, размещения на серверах или распространения в списках требуется предварительное специальное разрешение и / или плата. Запросить разрешения у permissions@acm.org. © 2018 Ассоциация вычислительной техники. 0360-0300/2018/09- ART96

\$15.00 <https://doi.org/10.1145/3197978>

(например, OpenMP, MPI, OpenCL и OpenACC). Кроме того, растущая сложность современных компиляторов и растущая озабоченность по поводу безопасности являются одними из наиболее серьезных проблем, с которыми сталкивается сообщество исследователей компиляции.

Компиляторы использовались в течение последних 50 лет [4, 115] для генерации машинно-зависимого ехе-вырезаемого двоичного файла из языков программирования высокого уровня. Разработчики компиляторов обычно разрабатывают этапы оптимизации для преобразования каждого сегмента кода программы с целью создания оптимизированной версии приложения. Оптимизации могут применяться на разных этапах процесса компиляции, поскольку компиляторы имеют три основных уровня: (1) *интерфейс*, (2) *промежуточное представление* (IR) и (3) *серверная часть*. В то же время оптимизация исходного кода вручную - утомительная задача. Оптимизация компилятора предоставляет автоматические методы преобразования кода. С этой целью оптимизация промежуточной фазы играет важную роль в показателях производительности. Включение параметров оптимизации компилятора (например, развертывание цикла, выделение регистров и т.д.) Может существенно улучшить несколько показателей производительности. В зависимости от целей, этими показателями могут быть время выполнения, размер кода или энергопотребление. Комплексный исследовательский подход к сопоставлению этих показателей также представляет собой сложную проблему [193].

Автонастройка [35, 256] обращается к автоматической генерации кода и оптимизации с использованием различных сценариев и архитектур. В нем разрабатываются методы автоматической оптимизации различных параметров для максимизации или минимизации выполнимости целевой функции. Исторически несколько оптимизаций выполнялись в серверной части, где *планирование, распределение ресурсов генерация кода* выполнены [56, 93]. Ограничения и ресурсы образуют линейную систему (ILP), которую необходимо решить. В последнее время исследователи демонстрируют возросшие усилия по внедрению интерфейсной и ИК-оптимизации. Два наблюдения подтверждают это утверждение: (1) сложность внутреннего компилятора требует исключительных знаний исключительно от разработчиков компилятора и (2) более низкие накладные расходы при модификации внешнего компилятора по сравнению с внутренними модификациями. Процесс ИК-оптимизации обычно включает в себя точную настройку параметров оптимизации компилятора с помощью формулировки многоцелевой оптимизации, которую может быть сложнее исследовать. Тем не менее, каждый подход имеет свои преимущества и недостатки и подлежит анализу в рамках их сферы применения.

Основной проблемой при выборе правильного набора оптимизаций компилятора является тот факт, что эти оптимизации кода зависят от языка программирования, приложения и архитектуры. Кроме того, слово оптимизация является неправильным - нет гарантии, что преобразованный код будет работать лучше, чем исходная версия. Фактически, агрессивные оптимизации могут даже снизить производительность кода, к которому они применяются [251]. Понимание поведения оптимизаций, предполагаемого воздействия на исходный код и взаимодействия оптимизаций друг с другом являются сложными проблемами моделирования. Это понимание особенно сложно, потому что разработчики компилятора должны учитывать сотни различных оптимизаций, которые могут быть применены на различных этапах компиляции. Проблема упорядочения фаз реализуется при рассмотрении порядка, в котором используются эти сотни оптимизаций. Существует несколько открытых проблем, связанных с областью оптимизации компилятора, которые не были должным образом решены [274]. Эти проблемы включают в себя поиск *какие* оптимизации использовать, *какие* набор параметров на выбор (например, размер тайлинга цикла, и т.д.), и в *каком порядке*

их применять для достижения наилучшего повышения производительности. Первые два вопроса создают проблему *выбора наилучших оптимизаций компилятора* и учета третий вопрос заключается в формировании *упорядочения этапов* проблемы оптимизации компилятора.

Проблема *упорядочивания по фазам* была открытой проблемой в области исследований компиляторов на протяжении многих десятилетий [19, 148, 151, 254, 262]. Неспособность исследователей полностью решить проблему фазового упорядочения привела к прогрессу в решении более простой задачи выбора правильного набора оптимизаций, но даже последняя еще не решена [21, 42, 60]. Процесс выбора правильных оптимизаций для фрагментов кода или ядер, обычно выполняется вручную, и последовательность оптимизаций строится с небольшим пониманием взаимодействия между предыдущим компилятором

оптимизации в последовательности. Задача построения эвристики для выбора правильной последовательности оптимизаций компилятора невыполнима, учитывая растущее число оптимизаций компилятора, интегрируемых в фреймворки компиляторов. Например, GCC имеет более 200 проходов компилятора, называемых компиляторами *Options*,¹ и LLVM-Clang и LLVM-опт каждой не более 150 трансформация *проходит*.² Кроме того, эти оптимизации нацелены на различные этапы процесса компиляции. Некоторые из этих проходов анализируют только доступ к данным, в то время как другие могут просматривать гнезда циклов. Большинство флагов оптимизации компилятора по умолчанию отключены, и разработчики компиляторов полагаются на то, что разработчики программного обеспечения знают, какие оптимизации должны быть полезны для их приложения. Разработчики компилятора предоставляют стандартные "именованные" уровни оптимизации, например, и так далее, чтобы ввести фиксированную последовательность -O1, -O2, -Os оптимизаций компилятора, которые в среднем обеспечивают лучшую производительность в наборе эталонных приложений, выбранных разработчиками компилятора. Однако, используя предопределенный оптимизационный, как правило, не является достаточно хорошим, чтобы принести максимально возможные для конкретных приложений производительность. Одним из ключевых подходов, недавно использованных в литературе для поиска наилучших оптимизаций для применения в данном приложении, является создание моделей прогнозирования с использованием различных классов машинного обучения (ML) [8]. Основное внимание в этом обзоре уделяется подходам, использующим машинное обучение для поиска наилучших оптимизаций для применения.

Вклад. В этом обзоре мы представляем обширный обзор методов и подходов, предложенных авторами для решения вышеупомянутых проблем. Мы выделяем более 200 недавних параметров, предложенных для автоматической настройки компилятора при использовании машинного обучения. Классификация в различных подполях осуществляется по нескольким репрезентативным признакам, показанным на рисунке 1. Насколько нам известно, первое применение машинного обучения, адаптированное для автоматической настройки компиляторов, было предложено ссылками [69, 143, 180]. Однако существовали и другие оригинальные работы, которые обсуждали проблемы и выступали в качестве движущей силы использования машинного обучения [34, 42, 64, 164, 191, 207, 254, 263, 269]. Таким образом, мы рассматриваем последние 25 лет исследований, связанных с автоматической настройкой компиляторов, поскольку они охватывают весь период времени, указанный в литературе.³ Кроме того, эта статья может быть использована в качестве связующего элемента для двух существующих опросов [33, 223] в поле оптимизации компилятора.

Организация. Мы проводим этот опрос следующим образом: мы начинаем с обзора различных методов сбора данных; последующим представлением предварительной обработки техники. Затем мы обсуждаем различные модели машинного обучения и типы целей, для которых строится модель. Наконец, мы подробно останавливаемся на различных целевых платформах, на которых применялись эти методы. Кроме того, для облегчения этого организационного процесса, изображенного на рисунке 1, мы приводим Цифру 2, для которого мы показываем пример фреймворка автоматической настройки, использующего компиляторы и машинное обучение. Подробнее о номенклатурах автоматической настройки мы поговорим в разделе 2.4. Раздел 2 обсуждаются мотивы и проблемы, связанные с исследованием оптимизации компилятора; за этим следует анализ пространства оптимизации для двух основных задач оптимизации в разделе 2.3. Мы рассмотрим существующие *characterizations* методов и классификацию в разделе 3. Кроме того, мы обсуждаем модели машинного обучения, используемые в разделе 4 и приведем полную классификацию различных методов прогнозирования, использованных в недавнем исследовании в разделе 5. Раздел 6 рассматриваются различные методы исследования пространства относительно того, как конфигурации оптимизации перемещаются в пространстве оптимизации. В разделе 7, мы обсуждаем различные целевые архитектуры и фреймворки компилятора, задействованные в процессе настройки. Мы включаем краткий обзор фреймворка компиляции Polyhedral наряду с другими широко используемыми фреймворками компиляции в раздел 7. Раздел 8 включает обсуждение влиятельных работ в данной области, классифицированных по их (1) результативности, (2) новизне, влиянию, которое они оказали на

¹<https://gcc.gnu.org/onlinedocs/gcc/Options-Summary.html>.

²<http://llvm.org/docs/Passes.html>.

³Мы с нетерпением ожидаем продолжения ежеквартального обновления текущего опроса в его филиале arXiv [20]³, и мы

призываем исследователей сообщать нам о своих новых связанных работах.

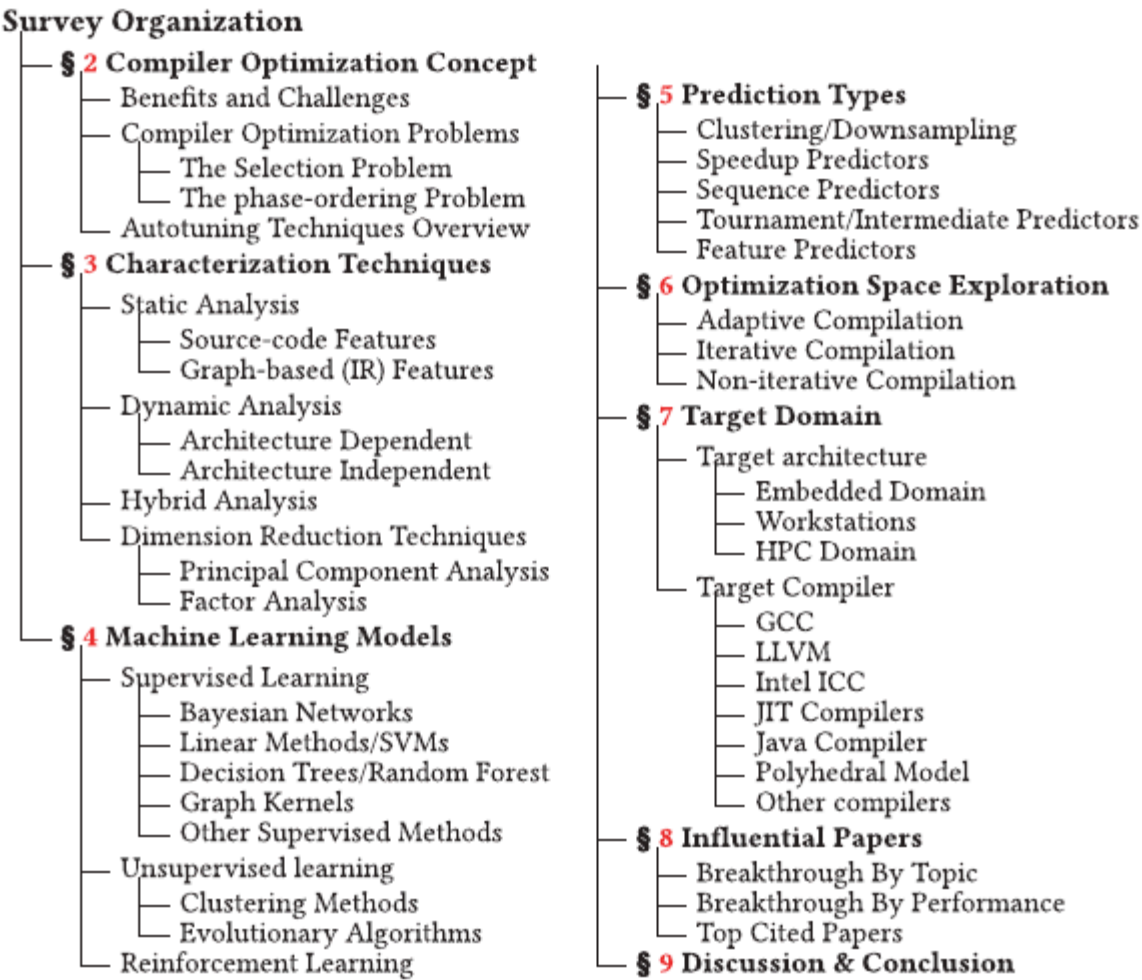


Рис. 1. Организация опроса по различным разделам.

последующая работа и (3) количество цитирований. Наконец, мы завершаем статью кратким обсуждением прошлых и будущих тенденций в методологиях автоматической настройки компиляторов с использованием машинного обучения.

Объем исследования. Мы распределяем исследования, проведенные в рамках этого опроса, по различным категориям, чтобы подчеркнуть их сходства и различия. Однако важно отметить, что представленное исследование имеет множество потенциальных классификаций в области машинного обучения и исследований компиляторов. Мы организуем опрос таким образом, чтобы все исследовательские работы, соответствующие определенному типу классификации, цитировались в рамках каждой классификации. Более того, в рамках каждой классификации мы выборочно отбирали наиболее заметные работы и более подробно описываем их вклад. В качестве примера достаточно Кавазоса и др.

[51] предложено использовать счетчики производительности для характеристики приложения.

Затем вектор функций можно использовать для построения моделей прогнозирования. Поскольку в этой работе использовался этот новый способ характеристики, он более подробно описан в Разделе 3.2.1. В соответствии с нашей политикой классификации эта работа также упоминается в других классификационных таблицах (тип прогноза, целевая платформа). Мы надеемся, что этот обзор будет полезен широкому кругу читателей, включая компьютерных архитекторов, разработчиков компиляторов, исследователей и технических специалистов.

2

ОПТИМИЗАЦИЯ КОМПИЛЯТОРА

2.1 Примечание по терминологии и показателям

Поскольку исследовательские работы, упомянутые в этом обзоре, принадлежат разным авторам, терминология может определяться локально и контекстуально. В результате получаются определения терминов, которые могут быть не совсем точными.

Таблица 1. Классификация, основанная на типе проблемы

Классы	Ссылки
Проблема выбора	[3, 6, 11, 15-17, 21, 22, 24, 27-29, 39, 42, 50-55, 61, 62, 67, 69, 70, 72, 78, 79, 83, 91, 92, 96, 98-101, 105-108, 116, 122, 123, 132, 135, 136, 142, 143, 152, 155, 157, 160, 162, 162, 163, 166, 169, 170, 178, 183, 195-198, 200, 201, 202, 206, 208-210, 217-219, 221, 235-238, 240, 243, 244, 248, 250, 253, 259, 267, 271, 273]
Проблема с фазовым упорядочением	[17-19, 25, 26, 28, 68, 111, 127, 146-149, 151, 153, 171, 184-187, 199, 212, 213, 251, 254, 262, 263]

определено для применения ко всем цитируемым публикациям. Мы разъясняем термины, используемые в этом обзоре, и соотносим их с обсуждаемыми публикациями. Область оптимизации компилятора обозначена как *автонастройка компилятора*, *Сборник, оптимизация кода, коллективная оптимизация, и оптимизация программы*. Чтобы сохранить ясность, мы не используем все эти термины, а вместо этого используем *оптимизирующие компиляторы* или *автонастройка компилятора*. Более того, в каждом классифицированном подразделе мы укажем другие номенклатуры, которые широко использовались, и нашу последующую ссылку.

2.2 Проблемы и преимущества оптимизации компилятора

Большая часть потенциального ускорения больше не сводится к увеличению тактовой частоты ядра процессора . Необходимы автоматические методы использования параллелизма и уменьшения зависимостей. Таким образом, оптимизация компилятора [191] позволяют пользователю влиять на сгенерированный код, не изменяя исходный высокоуровневый исходный код. Применение этих оптимизаций может привести к тому, что программа будет лучше работать на целевой архитектуре. Поскольку пользователь не может вручную настроить большую базу кода, необходимо внедрить автоматические методы. Более того, ручная настройка непереносима - преобразования, применяемые к коду, работающему на одной архитектуре, не гарантируют такого же увеличения производительности на другой архитектуре.

2.3 Проблемы оптимизации компилятора

Проблема взаимозависимости между этапами оптимизации компилятора характерна не только для области оптимизации компилятора. В традиционных оптимизирующих компиляторах были отмечены фазовые взаимосвязи между анализом потока и сверткой констант, а также между генерацией кода и регистрацией распределения ресурсов [156, 254]. В теории оптимизации "допустимый набор, пространство поиска или пространство решений" - это набор всех возможных точек (наборов значений переменных выбора) задачи оптимизации, которые удовлетворяют ограничениям задачи, потенциально включая неравенства, равенства и целочисленные ограничения" [239]. Оптимизация процесс обычно начинается с поиска стартового набора, а может-антов обычно обрезают и уменьшается (в зависимости от алгоритма/сценарии). Проблемы оптимизации компилятора могут быть разделены на одну из двух подобластей в зависимости от того, включаем ли мы (i) / отключаем набор оптимизаций (проблема выбора оптимизации) или (ii) изменяем порядок этих оптимизаций (проблема упорядочения фаз). Здесь мы кратко обсудим различные области оптимизации из двух. Таблица 1 классифицирует существующую литературу в зависимости от типа задачи оптимизации компилятора . В более поздних работах рассматривалась проблема выбора, поскольку проблема упорядочения фаз является более сложной исследовательской задачей для решения.

Проблема выбора наилучших оптимизаций компилятора. Несколько оптимизаций компилятора 2.3.1 проходы формируют последовательность оптимизации. Когда мы игнорируем порядок этих оптимизаций и сосредотачиваемся на том, следует ли применять оптимизации, мы определяем область действия проблемы как выбор наилучших оптимизаций компилятора. Предыдущие исследователи показали, что взаимозависимости ACM Computing Surveys, Том 51, № 5, статья 96. Дата публикации: сентябрь 2018 года.

и взаимодействие между включением или отключением оптимизации в определенной последовательности может существенно изменить производительность выполняемого кода, даже игнорируя порядок этапов [3, 21, 42].

Оптимизация пространства. Пусть $\mathbf{o}$ быть логическим вектором, имеющим элементы o_i как различные оптимизации компилятора. Элемент оптимизации o_i может быть определен как либо $o_i = 1$ (включен), или $o_i = 0$ (отключено). Последовательность оптимизации может быть задана вектором $\mathbf{o}$. Этот вектор является логическим и имеет n размеры:

$$|\Omega_{\text{Выбор}}| = \{0, 1\}^n \quad (1)$$

Для применения $\mathbf{a}$ оптимизируемый, n показывает количество анализируемых оптимизаций [15].

Пространство оптимизации задачи выбора имеет экспоненциальное пространство в качестве верхней границы. Например, достаточно, с $n = 10$, общее пространство состояний равно 2^n (1024) варианты оптимизации на выбор.

Различные варианты оптимизации масштабируются в зависимости от общего количества целевых приложений $\mathbf{a}$, дающий поиск в пространстве $A = \mathbf{a}$, где A это анализируемый набор наших приложений. Можно определить расширенную версию пространства оптимизации, переключив двоичный выбор (вкл. или выкл.) на вариант со многими вариантами выбора для каждой оптимизации компилятора. Уравнение (1) показывает случай, когда у нас есть нечто большее, чем просто бинарный выбор. Определенные оптимизации компилятора, такие как развертывание цикла и разбиение на листы, предлагают множество постоянных коэффициентов настройки, т.е. 4, 8, 1, 6, m . Кроме того, некоторые оптимизации могут принимать более одного параметра. Чтобы упростить представление уравнения, мы рассматриваем m как и их общее количество вариантов оптимизации, которыми располагает оптимизация компилятора. Мы ограничиваем это предлагаемое представление дискретизированными значениями. Непрерывные диапазоны могут быть аппроксимированы путем предварительного определения дискретного числа вариантов в пределах непрерывного диапазона значений; однако мы не нашли непрерывных вариантов с какими-либо компиляторами, используемыми в этой области исследования. Следовательно, мы имеем предыдущее уравнение в виде

$$|\Omega_{\text{Выбор, Расширен}}| = \{0, 1, \dots, m\}^n \quad (2)$$

Проблема упорядочения фаз. Проблема, общая для многофазных оптимизирующих компиляторов, заключается в 2.3.2

что идеального порядка фаз не существует. Этап оптимизации A , преобразует программу таким образом, что препятствует эффекту некоторых оптимизаций, которые в противном случае могли бы быть выполнены с помощью следующего прохода B . Если порядок двух фаз изменен, то фаза B выполняет оптимизации, которые могут лишить A о возможностях. Двойственность этой ситуации заключается в том, что две фазы открывают друг для друга новые возможности [156]. Разработчики компилятора должны учитывать порядок, в котором размещается и выполняется каждая фаза оптимизации. "Пара фаз оптимизации (состоит из множества проходов) может быть взаимозависимой в том смысле, что каждая фаза может извлечь выгоду из преобразования, произведенного другой" [156, 254].

Пространство оптимизаций. Последовательность оптимизации в фазовом порядке находится в факторном пространстве из-за наличия перестановок:

(3)

где n показывает количество анализируемых оптимизаций [15, 18]. Однако упомянутая граница представляет собой упрощенную задачу упорядочения фаз, имеющую фиксированную длину последовательности оптимизации и не требующую повторяющегося применения оптимизаций. Разрешение многократного применения оптимизаций и последовательность оптимизаций переменной длины расширяют пространство задач до $|\Omega| = n! \cdot \text{фазы}$

$$|\Omega_{\text{Фазы, Повторение, Переменная продолжительность}}| = \sum_{i=0}^n \frac{n!}{i!} \quad (4)$$

где n это количество изучаемых оптимизаций и i является максимально желаемой длиной для последовательности оптимизации. Даже имея разумные значения для n и m , сформированное пространство поиска оптимизации огромно. Например, предполагая n и m оба равны 10, что приводит к оптимизации пространство поиска из более чем 11 миллиардов различных оптимизационных последовательностей для идентификации с учетом каждой части

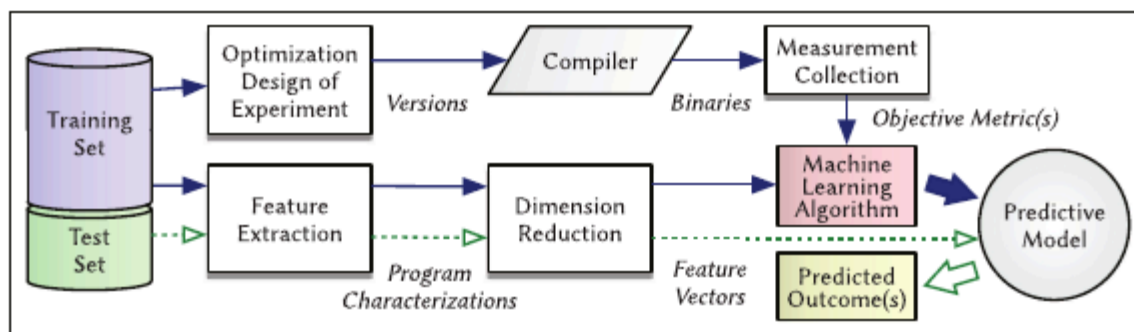


Рис. 2. Пример фреймворка автоматической настройки с использованием машинного обучения. (1)

Вверху: поток данных через различные компоненты фазы обучения, на которой индуцируется модель на основе характеристик приложений в рамках обучающего набора. (2) Внизу: поток данных через различные компоненты фазы тестирования, где уже обученная модель используется для прогнозирования результата для данного тестового приложения с приложениями, входящими в набор тестов.

из оптимизируемого кода. Проблема нахождения правильных фаз не имеет детерминированной верхней границы при неограниченной длине оптимизации [18, 156].

2.4 Обзор Методов автоматической настройки

Автонастройка относится к методологии, в которой существует некоторая модель (может быть эвристикой поиска, алгоритмом и т.д.). Которая определяет одну или несколько целей при минимальном взаимодействии с пользователем или вообще без него [266].

Рисунок 2 показан пример фреймворка автоматической настройки, использующего компиляторы и машинное обучение. В этом разделе будет рассмотрен каждый из компонентов этого примера фреймворка. В области компиляторов под автонастройкой обычно понимается определение стратегии оптимизации с помощью проекта эксперимента (DoE), ment (DoE-design of experiences, DoE-designing of experiences). 215], где определено пространство параметров настройки. Это пространство параметров создает множество версий данной программы ввода. Когда исследуется множество версий данной входной программы, это называется итеративным исследованием. Итеративное исследование может быть выполнено либо исчерпывающим образом, изучив все версии данного входного сигнала, либо выбрав подмножество пространства с помощью эвристики поиска. При использовании эвристики поискового исследования все пространство поиска может не быть исследовано. Это приводит к компромиссу между количеством экземпляров, используемых для построения модели (точность), и вычислительной сложностью, необходимой для построения модели (время).

Платформа автоматической настройки также должна иметь желаемую цель или результат, определяется в терминах производительности. Извлеченная информация из приложения и и передаются в алгоритм машинного обучения, на основе которого создается прогностическая. Извлеченная информация о приложении может быть чрезвычайно большой или иметь разл объема информации к извлеченным элементам обычно применяется та или иная форма уме размеров . Измере- Сьон сокращения необходима также для формирования последовательн различных приложений. Принципиальный компонентный анализ является одним из наиболее методов уменьшения размерности, о чем свидетельствуют проведенные исследования в это

3 МЕТОДЫ ОПРЕДЕЛЕНИЯ ХАРАКТЕРИСТИК ПРИЛОЖЕНИЯ

Для компьютерных архитекторов и разработчиков компиляторов понимание характеристик приложений, работающих в текущих и будущих компьютерных системах, имеет первостепенное значение

Таблица 2. Классификация, основанная на методах определения характеристик применения

Классификация		Ссылки
Статический	Исходный код	[3, 21, 28, 50, 52, 55, 66, 72, 78, 92, 96, 101, 105, 107, 108, 112, 115,
	Характеристики	151, 152, 155, 157, 160, 162, 166, 169, 170, 178, 183, 196, 198, 200, 202, 213, 217, 235, 236, 250, 270, 271]
Функции на основе графов		[52, 53, 67, 72, 78, 92, 96, 143, 155, 157, 166, 169, 183, 186, 196, 200, 202, 235, 237, 240, 250, 262]
Динамический (IR), зависящие от архитектуры,		[51, 61, 77, 79, 96, 98, 100, 136, 160, 169, 195, 196, 199-202, 217, 235, 237, 250, 253, 259]
	Независимые гибридные	[15, 18, 19, 21, 28, 61, 78, 199]
функции		[21, 28, 77, 155, 157, 169, 196, 200, 202, 217, 250]

Приложения машинного обучения к оптимизации компилятора требуют представления оптимизируемого кода. Таким образом, требуется инструмент для сканирования оптимизируемого кода с целью сбора его репрезентативных функций. Чтобы получить более точную модель, исследователи компиляторов пытались лучше понять поведение запущенных приложений и построить вектор функций, который наилучшим образом представляет функциональность пары. В общем, (1) производный вектор признаков должен быть репрезентативным в достаточной степени для своего применения, и (2) разные приложения не должны иметь один и тот же вектор признаков. Таким образом, построение большого, неэффективного вектора признаков замедляет - или даже останавливает - процесс ML и может снизить точность. В этом обзоре мы представляем различные технологии определения характеристик программы, используемые в справочной литературе по автоматической настройке компилятора: (i) статический анализ функций, (ii) динамический сбор функций и (iii) гибридный сбор функций, в котором используется комбинация двух предыдущих или других методов извлечения. Таблица 2 относится к таким классификациям.

3.1 Статический анализ

Статический анализ, или сбор статических функций, пытается собрать функции, которые нефункциональны для кода, выполняемого на данной архитектуре. Статический анализ включает в себя синтаксический анализ исходного кода во внешнем конце, промежуточном представлении (IR), серверной части или любой комбинации из трех. Сбор статических функций не требует выполнения кода и считается одним из самых надежных вариантов поддержки его использования. Мы кратко классифицируем функции исходного кода, использованные в предыдущих исследованиях.

3.1.1 *Функции исходного кода.* Функции исходного кода (SRC) - это абстракции некоторых выбранных свойств приложения ввода или текущего промежуточного состояния компилятора, когда другие оптимизации уже были применены. Эти функции варьируются от простой информации, такой как имя текущей функции, до значений параметров компилятора и порядка прохождения в текущем запуске компилятора. В литературе используется множество средств извлечения функций из исходного кода. Фурсин и др. предложили GCC Milepost. [101, 105], в качестве плагина к компилятору GCC для извлечения функций исходного кода.⁴

3.1.2 *Функции на основе графиков.* Представление на основе графиков делает зависимости данных и элементов управления явными для каждой операции в программе. "Графики зависимости от данных предоставили явное представление отношений определение-использование, неявно присутствующих в исходном коде" [86, 170].

График потока управления (CFG) [5] часто было представлением взаимосвязей потоков управления приложения. График зависимости программы явно представляет как основные отношения управления, так и существенные отношения данных (как показано на графике зависимости данных). Это

⁴http://ctuning.org/wiki/index.php/CTools:MilepostGCC:StaticFeatures:MILEPOST_V2.1

информация доступна без ненужной последовательности, присутствующей в графе потока управления [86]. Существует множество инструментов для извлечения графика потока управления ядром, функцией или приложением. MinIR [176], LLVM's Opt и IDA pro [80] доступны ли такие примеры этих инструментов.

Косеки и др. [143] использовали CFG и графики зависимостей, чтобы понять, какие факторы развертывания применимы к циклам. В соответствии с формулой для определения эффективности выполнения цикла P увеличивается монотонно, пока не насытится. Следовательно, их алгоритм никогда не выбирает направления, которые ведут к локальным максимальным точкам. Более того, они показали, что метод может найти точку, для которой развертывание выполняется наименьшее количество раз, поскольку он выбирает ближайшую точку насыщения, чтобы определить количество раз и направления, в которых выполняется развертывание цикла.

Паркетал. [198] представила новый фреймворк автоматической настройки компилятора, который использовал функции приложения на основе графов из его промежуточного представления (IR). Авторы использовали MinIR в форме SSA приложения для сбора графа потока управления основными блоками, включая направленные ребра, к которым у них есть предшественники и приемники. Для построения векторов признаков, необходимых в прогностическом моделировании, авторы использовали ядра графов кратчайших путей [46]. Метод сравнивал сходство кратчайшего пути двух приложений в каждом базовом блоке. После того, как были построены векторы признаков, была создана модель машинного обучения для прогнозирования ускорения данной последовательности. Наконец, авторы экспериментально оценили предложенный ими подход с использованием полиэдральной оптимизации и пакета PolyBench suite по сравнению с использованием функций исходного кода и динамических функций с несколькими бэкэндами компилятора.

3.2 Динамическая характеристика

Динамическая характеристика включает в себя извлечение счетчиков производительности (PC), которые используются для получения информации о том, насколько хорошо выполняется приложение. Эти данные могут помочь определить системные узкие места и дать рекомендации по точной настройке производительности приложения. Приложения также могут использовать данные счетчика производительности, чтобы определить, сколько системных ресурсов необходимо использовать. Например, достаточно, приложение с интенсивным использованием кэша данных может быть настроено за счет использования большего объема кэша. Здесь мы кратко опишем и классифицируем различные типы сбора счетчиков производительности. Обратитесь к таблице 2 для полной классификации по различным методам определения характеристик приложений.

Характеристика, зависящая от архитектуры. Последние процессоры предоставляют особую группу 3.2.1 регистры, которые позволяют собирать несколько измеримых событий, соответствующих их эффективности. Эти события могут быть измерены с минимальными сбоями в работе запущенного приложения и могут описывать несколько характеристик, таких как поведение кэша (попадания и промахи) и объем памяти [167]. Например, в работе Кавазоса и др. [51], в AMD Athlon было 4 регистра, которые могли собирать события счетчика производительности; однако можно было собрать 60 различных событий. "За счет мультиплексирования с использованием специальных регистров можно собирать от 4 до 60 типов событий за один запуск" [51]. Недостатком использования счетчиков производительности для характеристики приложений является их ограниченное повторное использование для других платформ. Собираемые показатели предназначены исключительно для платформы, на которой были собраны данные, что снижает переносимость платформы.

Существуют общедоступные инструменты для сбора таких показателей, предлагаемые ссылками [83, 169, 201], например, ПАПИ [182]. PAPI способен указывать стандартный интерфейс прикладного программирования (API) на различных архитектурах для сбора "переносимых" аппаратно-зависимых счетчиков производительности.

Мониторинг этих событий помогает процессу настройки путем построения корреляции между типом объектного кода и производительностью этого кода в целевой архитектуре. Триантафиллис и др. [251] использованный PFMon [128], еще один примечательный инструмент мониторинга производительности при выполнении программ для платформы Intel Itanium.

Кавазос и др. [51] предложили автономную модель, основанную на машинном обучении, которую можно использовать для предварительного определения хорошего набора последовательностей оптимизации компилятора. Их метод использует счетчики производительности для ACM Computing Surveys, Том 51, № 5, статья 96. Дата публикации: сентябрь 2018 года.

создайте вектор функций приложения и используйте их для прогнозирования хороших результатов оптимизации компилятора.

Авторы показали, что при использовании этого способа, они могли бы превзойти существующие статические техник-методическое, потому что они были в состоянии захватить памяти и кэша приложений.

3.2.2 *Архитектурно-Независимые Характеристики.* Информация, собранная из динамической характеристики, называемой вектором признаков, представляет собой компактное изложение динамического поведения приложения во время выполнения. Эта информация обобщает основные аспекты работы приложения производительность, т.е. количество промахов в кэше или использование единицы измерения с плавающей запятой [51]. Однако, информация может быть неточной или вводить в заблуждение, если приложение запущено на разных целевых архитектурах. Между различными целевыми объектами могут существовать различия, такие как размер кэша, порты выполнения и алгоритмы планирования. Счетчики, зависящие от архитектуры, несмотря на точность, они не могут быть использованы кроссплатформенно. Из-за этого ограничения исследователи предложили другой способ сбора данных о динамическом поведении, которые могут быть перенесены на другие платформы, если у них такая же архитектура набора команд (ISA), т.е. X86_64. Этот способ сбора функций известен как инструментирование и выполняется с использованием инструментов динамического анализа программ.

Intel Pin [165] является заслуживающей внимания платформой, которая позволяет создавать инструментальную коллекцию. *Закрепить* это ды-термодинамические бинарной инструментации инструмент, который анализирует двоичный так как он выполняет на данной архитектуры. Микроархитектура-независимый нагрузки характеристика приложения (следа) [121]isanex-достаточно одного Pintool в что способен собирать ряд особенностей программы во время выполнения. Эти собранные функции независимы от микроархитектуры и учитывают такие компоненты, как предиктор переходов, объем памяти и другие компоненты, существующие в различных архитектурах, а не конкретные программные счетчики, упомянутые ранее.

3.3 Гибридная характеристика

Метод гибридной характеристики состоит из комбинации ранее известных технологий таким образом, чтобы добавить больше информации об анализируемом приложении. В некоторых случаях [21, 200], гибридный выбор функций может более точно отражать поведение приложения, поскольку учитываются разные уровни извлечения функций.

Parketal. [200] использовал HERCULES, инструмент, основанный на шаблонах, для характеристики приложения [133, 134]. Этот метод определения характеристик подходит для выявления интересных закономерностей в приложении. Они использовали HERCULES для построения моделей прогнозирования, которые эффективно выбирают подходящие наборы последовательностей оптимизации компилятора для данного приложения.

3.4 Методы уменьшения размерности

В изученной литературе процесс уменьшения размеров важен по трем основным причинам: (i) он устраняет шум, который может помешать дальнейшему анализу, (ii) он сокращает размер и время обучения модели и (iii) улучшает понимание пространства объектов. В основном используются методы анализа главных компонент (PCA), [130], и Исследовательский факторный анализ (EFA) [113]. Сокращение пространства входных функций для алгоритма машинного обучения может привести к лучшим результатам обучения; однако используемый тип сокращения функций сам по себе может быть важным фактором [3, 21]. Например, PCA использовался в оригинальной работе, предложенной Ашури и др. [15], но позже авторы пересмотрели модель, используя EFA, и обнаружили явные преимущества [21]. Мы подробнее остановимся на этих методах в последних разделах обзора. Таблица 3 классифицирует использование каждой техники в изученной литературе.

Пусть y быть вектором характеристик, содержащим все функциональные данные выполнения приложения. Этот вектор представляет /переменные, учитываемые при статическом, динамическом или гибридном анализе. Давайте рассмотрим набор векторов признаков, представляющих профили характеристик приложения A состоящий из m векторы y Векторы признаков могут быть организованы в виде матрицы. PC m ряды и /столбцы. Каждый

Таблица 3. Классификация, основанная на методах уменьшения размеров

Классификация	Ссылки
Анализ главных компонент (PCA)	[3, 15, 18, 19, 21, 28, 29, 61, 78, 142, 198, 199, 202, 248]
Факторный анализ (FA)	[21]

вектор y (строка в P) включает в себя большой набор характеристик, полученных в результате анализа. Многие из этих характеристик приложения (столбцы матрицы P) сложным образом коррелируют друг с другом, в то время как другие характеристики могут не иметь видимой корреляции. "И PCA, и FA являются статистическими методами, направленными на определение способа представления y с более коротким вектором α при этом минимизируется потеря информации. Тем не менее, они полагаются на разные концепции для организации этого сокращения" [21, 113, 247]. В обоих случаях выходные значения выводятся путем применения уменьшения размера и больше не представляют определенный объект напрямую. В отличие от PCA, где компоненты задаются комбинацией наблюдаемых признаков, в EFA факторы представляют собой скрытый процесс, стоящий за генерацией признаков.

Анализ основных компонент. Цель состоит в том, чтобы определить краткое изложение y . Второй вектор, $3.4.1$, одинаковой длины $y(i)$ организуется некоторым извлеченным изменением переменной. p рассчитывается с помощью p линейная комбинация элементов v . Комбинация элементов v 's, дающая $y \cdot p$ принимается решение на основе анализа матрицы P . Все элементы в p ортогональны (некоррелированы) и отсортированы по их дисперсии. Из-за этих двух свойств первые элементы p (также называемые основными компонентами) несут большую часть информации о y . Сокращение может быть получено путем генерации вектора α это сохраняет только k наиболее важные основные компоненты в p . Обратите внимание, что основные компоненты в p они не предназначены для того, чтобы иметь значение; они используются только для суммирования вектора y в качестве подписи.

Факторный анализ. Это объясняет корреляцию между различными переменными в y . Коррелированный $3.4.2$ переменные в y вероятно, будут зависеть от одной и той же скрытой переменной в α . Взаимосвязь между α а наблюдаемые переменные получены с использованием метода максимальной вероятности, основанного на данных в матрице P . При принятии PCA каждая переменная в α имеет тенденцию состоять из всех переменных в y . Из-за этой характеристики трудно сказать, что представляет собой конкретный компонент в пространстве PCA. При внедрении EFA, компоненты α как правило, зависят от меньшего набора элементов в y которые коррелируют друг с другом. α представляет собой сжатое представление, где элементы в y которые коррелированы (т.е. Содержат одну и ту же информацию), сжимаются в уменьшенное количество элементов в α .

4 МОДЕЛИ МАШИННОГО ОБУЧЕНИЯ

Машинное обучение исследует изучение и построение методов и алгоритмов, которые способны извлекать определенные атрибуты из данных и делать на их основе прогнозы [177]. Существует множество типов и подполей машинного обучения, поэтому мы решили классифицировать их на основе трех широких категорий: (i) обучение под наблюдением, (ii) обучение без учителя и (iii) другие методы (включая подкрепляющее обучение, технику на основе графиков и статистические методы). Классификация представлена в Таблице 4. В каждом подразделе мы приводим обзор метода и краткое описание заметной связанной с ним работы. При адаптации приложения машинного обучения к компиляторам общая проблема оптимизации заключается в создании функции, которая "принимает в качестве входных данных кортеж $(F \cdot T)$, где F оптимизируется ли вектор характеристик собранных характеристик приложения" [199]; и T является одной из нескольких возможных последовательностей оптимизации компилятора, применяемых к этому приложению. Его выходной сигнал может представлять собой прогнозируемое значение ускорения в T , или, в зависимости от сценария, прогнозируемая ОПТИМИЗАЦИЯ ACM Computing Surveys, Том 51, № 5, статья 96. Дата публикации: сентябрь 2018 года.

Таблица 4. Классификация, основанная на моделях машинного обучения

Классификация		Ссылки
Обучение с надзором	Байесовская сеть	[15, 21]
	Линейные модели/SVMs	[18, 19, 21, 71, 199, 201, 217, 237]
	Леса	[41, 77, 92, 99, 99, 101, 105, 152, 162, 166, 178, 200, 253]
	Ядра графов	[138, 170, 171, 198]
	Прочее	[3, 11, 16, 19, 21, 21, 28, 28, 50-52, 69, 70, 72, 78, 79, 83, 91, 92, 96, 96, 98, 100, 101, 105, 107, 108, 116, 132, 136, 151, 152, 155, 162, 163, 183, 183, 197, 199, 201, 202, 206, 217, 221, 235, 235, 236, 236, 240, 248, 250, 253, 253, 259, 259]
Отключенное обучение	Кластеризация	[19, 29, 170, 171, 212, 248]
	Эволюционная Алгоритмы (ГАЗ, АККУРАТНО), NN)	[3, 6, 50, 54, 66, 67, 69, 70, 73, 78, 82, 110, 110, 122, 123, 142, 146-148, 151-153, 155, 157, 162, 171, 184, 186, 198, 202, 205, 209, 212, 235, 236, 238, 244, 259]
Обучение с подкреплением		[66, 172, 173]

последовательность T , применительно к исходному коду [199]. Существуют и другие типы прогнозов, которые мы подробно обсуждаем и классифицируем в разделе 5.

4.1 Контролируемое обучение

Контролируемое обучение - это подкласс машинного обучения, который занимается изучением функции на основе помеченных данных в обучающем наборе [76, 177]. Учащийся получает набор помеченных примеров в качестве обучающих данных и делает прогнозы для всех невидимых точек. Это наиболее распространенный сценарий, связанный с классификацией, регрессией, и Рейтинг проблемы.

Байесовские сети. Байесовские сети (BN) [95, 204] являются мощными классификаторами для вывода переменных распределения вероятностей, которые также могут быть двоичными, которые характеризуют определенный атрибут, такой как оптимальность последовательностей оптимизации компилятора. "Байесовская сеть - это прямой ациклический граф, узлами которого являются переменные, а ребрами - взаимосвязи между ними" [15, 21]. Вычисленные вероятности этих наблюдаемых переменных, классифицированных по узлам, могут быть дополнительно использованы в качестве входных данных для модели в качестве доказательства. Распределения вероятностей оптимизаций компилятора зависят от функций программы и взаимосвязи между различными применяемыми оптимизациями. Ашури и др. [15, 21] предложил байесовский сетевой подход для определения наилучшего компилятора оптимизации, подходящей для встроенного процессора. Этот подход использует статические, динамические и гибридные функции приложения в качестве доказательства для обученных сетей. Вместо того, чтобы использовать все обучающие данные, авторы предложили основывать свои обучающие данные только на тех экземплярах последовательностей компилятора, которые имеют наилучшее ускорение на 15% по сравнению со стандартным уровнем оптимизации GCC - O3

. Таким образом, BN

обучается только хорошим последовательностям оптимизаций и впоследствии может использовать оптимальные решения для невидимого приложения. Оценки проводились с помощью Sbench [97] и Полибенч [114, 211] сыютс и авторы показали, что использование BN с итеративной компиляцией превосходит GCC по эффективности -O2 примерно на 50%.

и -O3

Линейные модели и SVM. Линейные модели являются одним из самых популярных методов обучения под наблюдением 4.1.2

методы, которые будут широко использоваться исследователями при решении многих задач машинного обучения. "Линейная регрессия уха, ближайшего соседа и линейные пороговые алгоритмы, как правило, очень стабильны" [76].

Линейные модели - это алгоритмы, выходной классификатор которых не дает резких колебаний при незначительных изменениях в обучающем наборе. Более того, "SVM - это разновидность метода контролируемого машинного обучения, который может использоваться как для сценариев регрессии, так и для классификации". [201]. В SVMs существуют алгоритмы, которые могут адаптировать линейные методы к нелинейному классу задач, используя технику, называемую уловкой ядра [224]. Помимо построения гиперплоскости или набора гиперплоскостей в многомерном пространстве, SVM могут найти наилучшую гиперплоскость (так называемая кластеризация с максимальным запасом [137, 199]).

Санчес и др. [217] предложил SVMs изучить модели, чтобы сосредоточиться на автоматической настройке JIT-компилятора IBM Testarossa и плана компиляции build. Они использовали скалярные функции для построения векторов функций и применили схему обучения SVM для экспериментальной проверки качества своей модели с использованием сценариев с одной итерацией и 10 итераций на SPECjvm98

набор тестов.

Деревья решений и случайные леса. Бинарное дерево решений - это древовидное представление 4.1.3

разбиение пространства объектов. Деревья решений могут быть определены с использованием более сложных узловых вопросов, что приводит к разделению на основе более сложных поверхностей принятия решений [177]. Случайные леса, или случайные леса принятия решений, представляют собой метод коллективного обучения, который использует несколько методов обучения для обеспечения лучшего прогнозирования в целях регрессии и классификации. Случайные леса начинаются с построения множества деревьев решений во время обучения и вывода класса, который соответствует режиму классов (в случае классификации) или средствам классов (в случае регрессии). "Леса случайных решений исправляют привычку деревьев решений перестраиваться под свой обучающий набор" [76, 94].

Фрейзер и др. [92] предложил использовать машинное обучение для выполнения сжатия кода. Он использовал IR-структуру кодов для автоматического вывода дерева решений, которое разделяет промежуточный код представления в поток, который сжимается более эффективно. Они оценивали их код спор-Сьон подход с GCC и использовать коды операций, которые также могут помочь предсказать элементы операнда трансляция.

Monsifrot [178] обратился к автоматической генерации эвристики оптимизации для целевого процессора с помощью машинного обучения. Они использовали деревья принятия решений, чтобы изучить поведение оптимизации развертывания цикла в изучаемом коде, чтобы решить, следует ли выполнять развертывание на машинах UltraSPARC и IA-64.

Ядра графов. Ядра графов создают низкоразмерное представление данных по стоимости 4.1.4

функция, которая сохраняет свойства данных. "Использование функций ядра очень привлекательно, потому что входные данные не всегда должны быть выражены в виде векторов признаков" [143, 186, 198]. Графические ядра появляются как средство использования множества различных приложений машинного обучения в широком спектре приложений, начиная с обучения с частичным контролем [57] к кластеризации и классификации [48].

Паркетал. [198] предложил использовать ядра графов для характеристики приложения. Авторы использовали график потока управления программами. Вместо сглаживания информации о графике потока управления в вектор характеристик фиксированной длины, авторы создали матрицу подобия для использования в алгоритме SVM с использованием ядра графа кратчайшего пути [46]. Результат работы графа кратчайшего пути ядра - это оценка сходства между двумя графами. Они оценили этот подход, используя модель для как неитеративного, так и итеративного способа работы в двух многоядерных системах. GCC и Intel compiler были двумя внутренними компиляторами, когда код проходил через многогранный компилятор от источника к источнику. Наконец, они сравнили предложенную модель с использованием статических функций, полученных MilepostGCC, счетчики производительности и 5-реакции.

Другие контролируемые методы. В целях краткости мы решили классифицировать другие супер- 4.1.5 рассмотренные методы обучения в этом подразделе. К ним относятся нейронные сети, ANOVA, К-ближайший сосед, гауссовский процесс обучения. [177], и так далее.

Мосс и др. [180] показал процесс машинного обучения для построения эвристики для планирования инструкций, более конкретно, планирования линейного кода. Они использовали статические и ИК-характеристики базового блока с бенчмарком, чтобы экспериментально проверить свой подход, используя среднее геометрическое как функцию пригодности и перекрестную валидацию.

Кавазос и Мосс [52] использовал компилятор JIT Java и бенчмарки SPECjvm98 с набором правил модель индукционного обучения, чтобы решить, следует ли планировать инструкции. Они использовали контролируемое обучение, чтобы вызвать эвристику для прогнозирования, какие блоки следует запланировать, чтобы максимизировать выгоду, где индуцированная функция действует как бинарная классификация. Авторы экспериментально показали, что они могут добиться улучшения планирования по крайней мере на 90% для каждого блока, затратив при этом не более 25% необходимых усилий.

Ханеда и др. [116] представил статистическую модель для построения методологии, которая уменьшает размер пространства поиска оптимизации, тем самым позволяя использовать меньший набор решений для изучения стратегий. Они показали, что метод нашел единственную последовательность компилятора в наборе тестов SPECint95, которые могли бы работать в среднем лучше по сравнению со стандартным набором оптимизации GCC.

Турнавитис и др. [250] предложили методику, использующую обнаружение параллелизма на основе профиля, в которой они могли бы улучшить удобство использования функций исходного кода. Предлагаемый подход позволил авторам выявить и локализовать больший параллелизм в приложении с учетом обратной связи с пользователем, необходимой на заключительном этапе. Более того, авторы используют механизм прогнозирования, основанный на машинном обучении, который заменяет эвристику отображения для конкретной цели. Эта новая схема отображения позволила улучшить решения по отображению, будучи при этом более масштабируемой и практичной для использования на других архитектурах.

Намолару и др. [183] предложил общий метод систематической генерации числовых характеристик из приложения. Авторы реализовали подход в GCC. Этот метод не накладывает никаких ограничений на то, как логически и алгебраически объединять семантические свойства в числовые признаки. Следовательно, он статистически охватывает всю необходимую информацию, которую можно собрать из приложения. Они использовали статические функции MilePost GCC и MiBench для оценки своего подхода.

4.2 Неконтролируемое обучение

Неконтролируемое обучение - это задача машинного обучения по выводу функции для описания скрытой структуры из немаркированных данных. Поскольку примеры, предоставляемые учащемуся, не помечены, нет ошибки или сигнала вознаграждения для оценки потенциального решения [117, 177]. Обучение без учителя тесно связано с проблемой оценки плотности в статистике [231]; однако неконтролируемое обучение также включает в себя множество других методов, которые направлены на обобщение и объяснение ключевых особенностей данных, таких как использование эволюционных алгоритмов. В неконтролируемом обучении целей не существует, но среда, позволяющая оценивать модели, действительно существует. Среда может оценивать модель с некоторым значением, т.е. значением, переданным целевой функции модели.

Методы кластеризации. Одним из ключевых подклассов неконтролируемого обучения является кластеризация. Объединение помогает сократить количество проходов несвязанной оптимизации компилятора в значимые кластеры, которые соответствуют друг другу, т.е. нацелены на гнезда циклов или скалярные значения, или они должны следовать друг за другом в одной последовательности. Другая важность кластеризации и понижающей дискретизации заключается в сокращении пространства оптимизации компилятора, которое может быть на десятки тысяч порядков меньше (упомянуто в разделе 2.3).

Томсон и др. [248] предложил методологию сокращения времени обучения автоматической настройки на основе машинного обучения. Они предложили использовать метод кластеризации, а именно Густафсона Кесселя

алгоритм [32], после применения процесса уменьшения размеров. Они оценили подход к кластеризации с помощью набора тестов EEMBCv2 и показали семикратное сокращение времени обучения с помощью предлагаемого подхода.

Ашури и др. [16, 29] разработал набор инструментов совместного проектирования аппаратного и программного обеспечения для исследования пространства проектирования компилятора совместно с микроархитектурными свойствами VLIW-процессора. Авторы использовали кластеризацию для получения четырех хороших аппаратных архитектур с последующим смягчением выбора перспективной оптимизации компилятора с помощью статистических методов, таких как тест Крускала-Уоллиса и оптимальная по Парето фильтрация. (Этот метод также связан со статистическими методами. Обратитесь к разделу 4.1.5.)

Мартинс и др. [170, 171] решил проблему упорядочения фаз с помощью метода отбора на основе кластеризации для группировки функций, имеющих сходство. Авторы использовали ДНК-кодирование, в котором программные элементы (например, операторы и циклы при детализации функций) кодируются в последовательности символов, за которыми следует вычисление матрицы расстояний и древовидное построение набора оптимизаций. Наконец - то, они применили методы оптимизации, уже включенные в пространство оптимизации, для измерения ускорения исследования по сравнению с самыми современными методами, такими как генетический алгоритм.

Эволюционные алгоритмы. Эволюционные алгоритмы основаны на биологической эволюции 4.2.2 например, процесс естественного отбора и мутации. Возможные решения в пространстве оптимизации играют роль особей в популяции. Обычной практикой определения качества решений является использование функции соответствия, такой как ускорение выполнения. Эволюция совокупности происходит после повторного применения функции соответствия. [177]. Здесь мы кратко упомянем некоторые из наиболее известных методов, используемых в литературе.

Генетический алгоритм (GA) - это метаэвристический алгоритм этого класса, который может быть сопряжен с любым другим методом машинного обучения или использоваться независимо. Примечательной быстрой эвристикой GA является NSGA-II (генетический алгоритм сортировки без доминирования II). [75], который является популярным методом для многих многоцелевых задач оптимизации и нашел множество применений, в основном в области компьютерной архитектуры. [168, 229]. Показано, что NSGA-II снижает вычислительную сложность классических алгоритмов GA.

Другой интересной эволюционной моделью является нейроеволюция расширяющихся топологий (NEAT) [234]. Они оказались мощной моделью для изучения сложных задач, поскольку способны изменять топологию сети и вес параметров, чтобы найти наиболее сбалансированную функцию пригодности. NEAT, в частности, также использовался во многих известных недавних исследовательских работах [66, 151, 152]. В этом разделе обобщены несколько заметных исследовательских работ, в которых использовались эволюционные алгоритмы.

Купер и др. [69, 70] решена проблема размера кода сгенерированных двоичных файлов с помощью генетических алгоритмов. Результаты этого подхода были сравнены с итерационным алгоритмом, генерирующим фиксированную последовательность оптимизации, а также со случайной частотой. Проведя сравнение, авторы пришли к выводу, что, используя свой GAs, они могли бы разработать новые фиксированные последовательности оптимизации, которые, как правило, хорошо работают для уменьшения размера двоичного кода.

Knijnenburg et al. [142] предложил итеративный подход к компиляции для решения проблемы размера выборки факторов разбиения на листы и развертывания независимо от архитектуры. Авторы оценивали их подход через ряд государственно-оф-искусства итерационные методы компиляции, например, сим-ОПы отжига и газа, и родной Fortran77 или g77 компилятор, позволяющий оптимизировать для

Матрица-умножение матрицы ($M \times M$). Умножение матрицы на вектор ($M \times V$) и прямая дискретизация Косинусного преобразования.

Агаков и др. [3] адаптировал ряд моделей для ускорения исследования итеративного вычислительного пространства. Методология использует оракул цепочки Маркова и независимый оракул распределения вероятностей с идентичным распределением (IID). Авторы добились значительного ускорения, направляя свою итеративную компиляцию с использованием этих двух моделей при тестировании на невидимых приложениях. Когда ACM Computing Surveys, Том 51, № 5, Статья 96. Дата публикации: сентябрь 2018 года.

прогнозируя наилучшую оптимизацию для невидимого приложения, они используют класс ближайшего соседа. Во-первых, он идентифицирует обучающее приложение, имеющее наименьшее евклидово расстояние в пространстве объектов векторное пространство (полученное с помощью PCA). Затем он изучает распределение вероятностей наилучших оптимизаций компилятора для этого соседнего приложения либо с помощью цепочки Маркова, либо с помощью IID-модели..... Полученное распределение вероятностей затем используется в качестве прогнозируемого оптимального распределения для нового приложения. Оракул с цепью Маркова превзошел оракул IID, за ним последовала методология RIC, использующая равномерное распределение вероятностей.

Leather et al. [155] использовал основанный на грамматической эволюции подход с генетическим алгоритмом для описания пространства признаков. Авторы показали, что выбор правильных функций для компилятора может привести к существенному повышению производительности эвристики машинного обучения. Они описали пространство, образованное объектами, с помощью грамматики, а затем сгенерировали из него множество объектов. Эти созданные объекты затем используются в прогностического моделирования для поиска ОП- timization пространства, используя GCC и Mediabench и экспериментально проверить предлагаемый подход на Pentium машины.

Кулкарни и соавт. разработали два подхода к решению обеих задач выбора оптимизации. [152] и фазовый порядок [151]. Подход к выбору хороших проходов компилятора выполняется с использованием АККУРАТНЫХ и статичных функций для настройки компилятора Java Hotspot в соответствии со спецификационными бенчмарками Java (используя два бенчмарка для обучения и два для тестирования). Авторы использовали NEAT для обучения деревьев принятия решений для вызывающего абонента и вызываемого абонента о том, следует ли вставлять. При решении проблемы упорядочения фаз они предложили метод прогнозирования промежуточного ускорения, который использовал статические характеристики текущего состояния изучаемого приложения для запроса модели и побуждения использовать текущую наилучшую оптимизацию. Таким образом, итеративно последовательность компилятора формируется на основе приложения.

Пурины и др. [212] определил подход, основанный на машинном обучении, для уменьшения выборки набора оптимизационных последовательностей компилятора в рамках LLVM и применил машинное обучение для обучения модели.

Авторы представили алгоритм кластеризации для кластеризации последовательностей на основе матрицы подобия путем вычисления евклидова расстояния между двумя векторами последовательности. В экспериментальной оценке они также упомянули наиболее частые этапы оптимизации с их функцией пригодности (ускорение выполнения). Позже Ашури и др. [19] использовал такую матрицу подобия для LLVM - о3 проходы, чтобы ускорить исследование или рекомендацию предсказанных последовательностей.

4.3 Обучение с подкреплением

Обучение с подкреплением (RL) - это область машинного обучения, которая не может быть классифицирована как контролируемая или неконтролируемая. Она вдохновлена бихевиористской психологией и использует понятие вознаграждений или наказаний, чтобы программный агент взаимодействовал с окружающей средой и максимизировал свое совокупное вознаграждение. Интересное отличие RL заключается в том, что этапы обучения и тестирования взаимосвязаны [177]. RL использует марковский процесс принятия решений (MDP) [124] для адаптации и взаимодействия с окружающей средой. В этом разделе мы представили работы, выполненные с помощью RL в полевых условиях.

Макговерн и др. [172] представлены два метода построения планировщика команд с использованием развертываний, улучшенный поиск по методу Монте-Карло [246], и обучение с подкреплением. Авторы показали, что комбинированный подход к обучению с подкреплением и развертыванию может превзойти коммерческий планировщик Compaq по оценочным показателям из пакета SPEC95 suite.

Кунс и др. [66] используется NEAT в качестве инструмента обучения с подкреплением для поиска хороших мест для обучения для пограничной архитектуры. Авторы показали, что их подход может превзойти самые современные методы, использующие имитацию отжига для поиска наилучшего размещения.

5 ТИПОВ ПРОГНОЗИРОВАНИЯ

Основными классами типа прогнозирования являются: (i) для прогнозирования правильного набора оптимизаций компилятора, которые будут использоваться, (ii) для прогнозирования ускорения последовательности оптимизации компилятора без фактического выполнения

Классификация	Ссылки
Кластеризация / Понижающая дискретизация	[19, 29, 111, 122, 142, 170, 171, 185, 186, 212, 248]
Ускоряющее	[18, 19, 21, 28, 50, 67, 77, 78, 96, 99, 100, 136, 152, 155, 199-202, 235, 236, 259]
Прогнозирование Последовательности Компилятора	[7, 6, 14, 16, 21, 28, 29, 38, 41, 42, 50, 51, 52, 53, 54, 55, 61, 67, 69, 70, 79, 83, 91, 98, 99, 101, 105, 106, 107, 108, 116, 122, 123, 132, 142, 146-149, 151, 160, 162, 163, 166, 169-171, 178, 183-185, 196, 198, 201, 202, 206, 208, 209, 212, 213, 217, 221, 235-237, 240, 248, 250, 253, 259]
Турнир/ Промежуточный	[18, 151, 199, 201, 202, 251]
Функция Предсказания	[66, 73, 77, 157, 202, 253]

последовательность, (iii) для прогнозирования правильного набора функций, которые будут использоваться для характеристики приложения, (iv) прогноз промежуточного ускорения (турнира) и (v) сокращение пространства поиска за счет уменьшения выборки оптимизаций. Таблица 5 показана классификация соответствующей литературы.

5.1 Кластеризация и понижающая дискретизация

Кластеризация, или кластерного анализа, является задача исследования сходства между набором вари- ременных таким образом, что подобные переменные могут быть помещены в одну группу, называемую кластером. Группировка чаще всего используется в области интеллектуального анализа данных и является распространенным методом статистического анализа данных. Кластерный анализ используется во многих областях, включая неконтролируемое машинное обучение, распознавание образов и автоматическую настройку компилятора [177]. Чтобы приблизиться к разумным решениям задач комплексной оптимизации, необходимо сократить пространство поиска оптимизации. Исследователи пытаются найти способы уменьшить огромный размер пространства оптимизации на порядки. Этот метод, известный как понижающая дискретизация, использовался во многих недавних работах. Мы упоминаем эти документы в Таблице 5.

Ашури и др. [19] представил предиктор ускорения полной последовательности для задачи упорядочения фаз, который быстро сходится к оптимальным точкам и превосходит как стандартные уровни оптимизации, так и самые современные *Рейтинг* подход [198, 199]. Авторы применили метод кластеризации ко всем этапам оптимизации, доступным в процедурах LLVM - O3 и обучили свою модель, используя полученный результат . Неосуществимо большое пространство упорядочения фаз сводится к довольно большому, но исследуемому с помощью итеративной компиляции и метода машинного обучения. Авторы показали, что их подход может превзойти самый высокий уровень оптимизации LLVM и другие существующие предикторы ускорения всего за несколько предсказаний.

5.2 Прогнозирование ускорения

Прогнозное моделирование ускорения - это процесс построения, тестирования и валидации модели для прогнозирования ненаблюдаемого временного результата. Модель строится на основе характеристики состояния. Характеризуемое состояние - это оптимизируемый код, а прогнозируемый результат соответствует метрике ускорения, рассчитанной путем нормализации времени выполнения текущей последовательности оптимизации на время выполнения базовой последовательности оптимизации. "Общая форма предиктора ускорения заключается в построении функции, которая принимает в качестве входных данных кортеж (F, T) , где F оптимизируется ли собранный вектор функций приложения" [199]. T может быть одним из

множество возможных последовательностей оптимизации компилятора в пространстве проектирования. Выходными данными модели являются прогнозируемое значение ускорения, которое последовательность T достигается при применении к исходному коду приложения в его исходном состоянии. Этот тип прогнозирования является одним из наиболее широко используемых среди исследователей, и здесь мы упоминаем некоторые из его примечательных применений.

Дубах и др. [78, 79] предложил предиктор ускорения, основанный на особенностях исходного кода оптимизированных приложений. Авторы использовали статические функции из инфраструктуры компилятора SUIF [266] для VLIW и сравнил результат с альтернативными предикторами, не основанными на признаках, такими как среднее предикторы, предикторы, основанные на кодировании последовательности, и предсказатели, основанные на реакции.

Парк и др. [198-202] предложил несколько методологий прогнозного моделирования для решения проблемы выбора правильного набора оптимизаций компилятора. В справке [198], авторы использовали графики потоков управления (CFG) с обучением ядра графа для построения модели машинного обучения. Сначала они построили CFG с помощью компилятора LLVM и преобразовали CFG в графики кратчайшего пути (SPG) с использованием алгоритма Флойда-Уорсхолла. Затем они применяют метод кратчайшего ядра графа [46] чтобы сравнить каждую из возможных пар SPG и вычислить оценку сходства двух графиков. Рассчитанные ими баллы сходства для всех пар сохраняются в матрице и напрямую загружаются в выбранный алгоритм машинного обучения, в частности в SVM в их работе. Позже, в отношении [199], вместо использования аппаратно-зависимых функций авторы использовали статические функции из исходного кода в многогранном пространстве и предсказывали производительность каждой высокоуровневой сложной последовательности оптимизации с помощью обученных моделей. В более поздней работе [200], они использовали пользовательские шаблоны в качестве программных функций, названных HERCULES [133], для получения произвольных шаблонов, поступающих от пользователей. Они сосредоточились на определении шаблонов, связанных с циклами: количество циклов, имеющих доступ к памяти, имеющих зависимости, переносимые циклом, или определенные типы зависимостей от данных. В этих работах используются функции статической программы, в основном фокусирующиеся на циклах и комбинациях команд.

5.3 Предсказание последовательности компилятором

Предиктор последовательности компилятора - это тип модели прогнозирования, которая выводит наилучший набор проходов или последовательностей компилятора для применения в данном приложении. Характеристика приложения передается в эту модель, и модель индуцирует предсказание набора проходов компилятора $\sigma \in \Omega$ подать заявку на данное заявление. Цель может быть настраиваемой - она может максимизировать свою производительность или выполнять другие цели, такие как размер кода или энергопотребление [15, 21, 201]. Этот конкретный тип прогнозирования вызвал наибольший интерес среди исследователей, и мы видели большую часть работ, посвященных этому. Здесь мы обсудим несколько заметных вкладов.

Кавазос и др. [51] исследовал проблему выбора правильного набора оптимизаций компилятора с помощью динамических характеристик анализируемого приложения. Авторы использовали логистическую регрессию для построения модели и показали, что использование их подхода может превзойти существующие методы, используемые для статических функций кода, и быстро достичь достижимого ускорения.

Ашури и др. [15, 21] предложил байесовский сетевой подход (BN), который опирался либо на статический, динамический, либо на гибридный вектор характеристики приложения. Модель BN могла бы впоследствии индуцировать вероятностную модель. Авторы объединили свою модель с итеративной компиляцией, чтобы получить хорошие последовательности компиляции, и экспериментально показали, что они могут превзойти самые современные модели.

5.4 Турнирный и промежуточный прогноз

Турнирный прогноз, предлагаемый по ссылке [201], предсказывает двоичную классификацию между двумя входными последовательностями оптимизации. Этот тип предиктора может соответствующим образом ранжировать последовательности компилятора и выбирать лучшие из них. Однако промежуточный предиктор ускорения (также называемый моделированием на основе реакции [50]) [18, 151] - это модель прогнозирования, которая имеет тенденцию итеративно предсказывать ускорение

о текущем состоянии оптимизируемого приложения. Характеристики приложений в каждом состоянии вместе с последовательностью компиляции T служат входными данными, и модель предсказывает ускорение T должен достичь. Поскольку промежуточные предикторы ускорения ведут себя более или менее так же, как турнирные предикторы, поскольку оба работают над индивидуальной оптимизацией одновременно, которая должна быть применена к текущему состоянию, мы решили сгруппировать эти два под одной классификацией. Подход с промежуточной последовательностью требует оптимизации нескольких профилей выполнения приложения; следовательно, он, как правило, медленнее по сравнению с другими методами. Тем не менее, был продемонстрирован эффективный метод конкретно для решения проблем упорядочения фаз [18, 151].

Парк и др. [201] предложили предсказатели турнира, чтобы определить, следует ли применять оптимизацию в качестве немедленной оптимизации. Park et al. оценил три модели прогнозирования с использованием программных счетчиков (PC), полученных PAPI [182]: последовательность, ускорение и турнир в нескольких бенчмарках (Polybench, NAS и т.д.) С использованием компилятора Open64. Они показали это во многих случаях предсказатели турниров могут превзойти другие методы.

Кулкарни и Кавазос. [151] разработал промежуточный предиктор ускорения с использованием NEATs, который выбирает наилучший порядок прохождения оптимизации компилятора итеративно на основе его текущего состояния. Они оценили свой подход с использованием динамических компиляторов JAVA Jikes и наблюдали в среднем ускорение на 5-10% при настройке различного порядка следования фаз.

Ашури и др. [18] продемонстрировал методику прогнозирования промежуточного ускорения, полученную путем оптимизации из конфигурационного пространства с учетом текущего состояния приложения. Авторы использовали динамические особенности приложения для своей модели прогнозирования, в отличие от работы Кулкарни и Кавазоса [151]. Они также использовали значения ускорения между временами выполнения программы до и после процесса оптимизации в качестве своей функции пригодности и определили эвристику для обхода пространства оптимизации.

5.5 Прогнозирование функций

Построение модели на основе эмпирических данных должно требовать минимального вмешательства пользователя. Точность прогнозов в значительной степени зависит от типа функций и характеристик, собираемых из приложения в процессе оптимизации. Следовательно, модели прогнозирования функций выгодно использовать для изучения наиболее перспективных функций, влияющих на производительность. В ходе этого процесса решающее значение имеет выбор правильного набора функций для характеристики приложения.

Васвани и др. [253] использовал эмпирические данные для построения модели, чувствительной к параметрам микроархитектуры и последовательностям оптимизации компилятора. Предлагаемая модель автоматически извлекает взаимосвязь из данных и строит автоматическую эвристику. Авторы оценили свой подход с использованием компилятора GCC и в среднем достигли улучшения производительности, более чем на 10% превышающего самые высокие стандартные уровни оптимизации, доступные в GCC.

Li et al. [157] использовал оптимизацию, основанную на машинном обучении, ориентированную на обработку объектов. Авторы адаптировали этот метод на основе анализируемого приложения и помимо предопределенных функций исходного кода. Они разработали подход к автоматической генерации нескольких функций приложения, которые будут переданы конструктору модели с использованием шаблона. Кроме того, они наблюдали колебания значений функций при использовании различных оптимизаций компилятора и уменьшили это, разработав средство извлечения функций для итеративного извлечения требуемых функций во время выполнения для подачи в оптимизационную модель.

Динг и др. [77] предложил структуру автоматической настройки, способную включать различные входные данные в двухуровневый подход для решения проблемы чувствительности входных данных. Они использовали язык и компилятор Petabricks [10] и использовали методику обучения с учетом входных данных, чтобы различать входные данные. В ходе работы было сгруппировано пространство и выбран центр тяжести для автоматической настройки (i) для определения набора конфигураций для каждого класса входных данных и (ii) для определения производственного классификатора, который позволил им эффективно определять наилучшую оптимизацию ориентиров для использования при новых входных данных. ACM Computing Surveys, Том 51, № 5, статья 96. Дата публикации: сентябрь 2018 года.

Таблица 6. Классификация, основанная на методах исследования космоса

Классификация	Ссылки
Адаптивная Компиляция	[6, 10, 11, 41, 51, 55, 62, 67, 69, 70, 77, 79, 83, 91, 96, 98, 99, 101, 105, 107, 108, 116, 122, 123, 146, 151, 155, 162, 163, 166, 169, 183, 189, 196, 197, 206, 217, 219, 235-237, 240, 253, 260, 267]
Итеративная Компиляция	[1, 3, 6, 10, 11, 15, 16, 16, 18, 19, 21, 28, 29, 38, 39, 42, 50, 51, 53-55, 58, 61, 67, 69, 70, 77-79, 83, 91, 96, 98-101, 105-108, 116, 122, 123, 135, 136, 142, 143, 146-153, 155, 160, 162, 163, 163, 166, 169-171, 183-186, 189, 196-202, 206, 208-210, 217, 219, 221, 235-237, 243, 245, 248, 250, 253, 259, 267, 270, 271]
Неитеративный	[92, 167, 209, 210, 213, 235, 254, 262, 263]

6 ОПТИМИЗАЦИЯ МЕТОДОВ ИССЛЕДОВАНИЯ КОСМИЧЕСКОГО ПРОСТРАНСТВА

Как упоминалось ранее в разделе 2.3, для обхода большого пространства оптимизации компилятора, создаваемого различными комбинациями оптимизаций, требуется надлежащая стратегия исследования. Итеративная компиляция и генетические алгоритмы являются одними из наиболее упоминаемых стратегий в литературе. В более широкой перспективе стратегии определяются желаемым типом расширения пространства оптимизации. Исследование пространства проектирования (DSE) - это деятельность по изучению альтернатив проектирования перед внедрением. Исторически он использовался при проектировании на системном уровне, но методологии могут быть адаптированы и для использования при исследовании на уровне компилятора [29]. DSE помогает определять и следовать политике планирования, которая предусматривает генерацию кандидатов в пределах определенного или всего пространства оптимизации и доказала свою полезность для многих задач оптимизации [145]. Как правило, различные приложения могут предъявлять разные требования к энергопотреблению и производительности. Основная цель этого этапа - эффективно отслеживать и настраивать параметры разведки. [192, 194]. В этом разделе мы классифицируем различные стратегии исследования, используемые исследователями в литературе для решения этой проблемы.

Таблица 6 представляет нашу детальную классификацию различных связанных работ на основе типа исследования.

6.1 Адаптивная компиляция

Адаптивная оптимизация [260], также известная как оптимизация с учетом профиля, представляет собой метод, при котором пространство оптимизации исследуется на основе результатов функций соответствия, например, времени выполнения для профилирования исполняемого файла, и динамически изменяет / перекомпилирует определенные сегменты приложения в процессе оптимизации. Профилирование предоставляет достаточно возможностей, чтобы компилятор мог решить, какую часть кода следует перекомпилировать. Адаптивный компилятор размещается между компилятором "точно в срок" (JIT) и интерпретатором инструкций. Как предлагается в ссылке [67], адаптивный компилятор может извлечь выгоду из цикла обратной связи компиляция-выполнение для выбора наилучшей оптимизации, которая удовлетворяет целям сценария. Следующие работы посвящены современному состоянию и практике профильной оптимизации.

Купер и др. [67, 70] разработал адаптивный компилятор ACME и пользовательский интерфейс для управления процессом перекомпиляции и изучения различных оптимизаций компилятора. Авторы представили виртуальное выполнение - метод, позволяющий уменьшить параллельное выполнение оптимизируемого кода. При однократном запуске приложения этот метод помогает вести учет информации, необходимой для оценки производительности различных последовательностей оптимизации компилятора во время выполнения без необходимости повторного запуска приложения. Позже этот метод был назван ускоряющим прогнозированием [18, 151, 201].

Фурсин и Коэн [99] построил итеративный и адаптивный фреймворк компилятора, ориентированный на СПЕЦИФИКАЦИЮ benchmark suite, используя модифицированную версию GCC. Авторы также разработали прозрачный

база, которая повторно все составитель программы процедуры анализа из программы трансформационной базе, чтобы избежать дубликатов во внешних инструментов оптимизации.

6.2 Итеративная Компиляция

В информатике итерационный метод - это последовательность приближенных процедур, которые применяются для дальнейшего повышения качества решения задачи. Итеративная компиляция является наиболее широко используемым методом исследования в области оптимизации компилятора. Многие недавние работы сочли этот метод интересным и успешным либо (i) сам по себе [42, 143], (ii) в сочетании с методами машинного обучения [3, 21, 51], или (iii) в сочетании с другими методами поиска и метаэвристики, такими как выборочное исследование [42] и методы DSE [170, 185].

Бодин и др. [42] исследовал ранний путь к анализу применимости итерационных методов поиска в оптимизации программ. Авторы показали, что итеративная компиляция применима ко встроенным приложениям, где первичные затраты окупаются за счет изготовления и масштабного распространения. Встроенное приложение обычно работает с меньшим количеством параметров, поэтому стоимость итеративной компиляции оправдана при небольшом количестве параметров. Они использовали обратную связь по профилю в виде времени выполнения и для уменьшения объема пространства на ограниченных этапах оптимизации. В этой работе авторы исследовали параметры развертывания, разбиения на листы и заполнения. Позже другие исследователи, вдохновленные итеративным подходом, исследовали другие параметры оптимизации для скалярной, вложенной в цикл, и других оптимизаций [16, 29, 83, 106, 139-141, 201, 209].

Триантафиллис и др. [251] предложил универсальный оптимизатор, который использовал практическую итеративную компиляцию фреймворк под названием Optimization-Space Exploration (OSE). Их подход предполагал использование знаний разработчика компилятора для сокращения и исключения нескольких параметров конфигурации в пространстве проектирования. Операционная система обладала степенью детализации функций и могла применять различные оптимизации к одним и тем же сегментам кода. Они использовали итерационный метод для выбора следующей оптимизации компилятора, которая будет использоваться, на основе текущего состояния оптимизируемого приложения. Подклассификация этого подхода позже была названа промежуточным прогнозированием ускорения [18, 151].

6.3 Неитеративная компиляция

В отличие от итеративной компиляции, неитерационный подход пытается представить глобальную оптимизацию подход для класса задач оптимизации компилятора. В последнее время наблюдается меньше работ, посвященных проблемам оптимизации компилятора с использованием этого метода [156]. Как мы отмечали в разделе 1, движущей силой к использованию методов аппроксимации, таких как итеративная компиляция и машинное обучение, была неспособность исследователей решить проблему упорядочения фаз, используя простые не-итерационные подходы. Таким образом, эта ветвь автоматической настройки компилятора подверглась дальнейшему исследованию. Сообщество по многогранной компиляции привлекло внимание во многих интересных направлениях и представляет собой ортогональный подход к оптимизации компиляторов. Мы рассматриваем многогранную компиляцию в разделе 7.

Вегдал и др. [254] использовали постоянное развертывание для создания кодовых последовательностей, которые могут быть более сжаты в горизонтальной целевой архитектуре. Аксиома, раскрывающая константу, заменяет константу на постоянное выражение равного значения. Цель состоит в том, чтобы использовать константы, которые жестко запрограммированы в микромаchine, заменяя трудно генерируемые константы выражениями, включающими только жестко запрограммированные константы.

Уитфилд и др. [263] предложил структуру, которая исследует использование спецификаций Евангелия [261] для повышения производительности оптимизируемого кода. Авторы внедрили инструмент под названием Genesis, который был способен преобразовывать коды на основе спецификаций Евангелия. Они продемонстрировали преимущества предлагаемого фреймворка посредством бинарного (включающего или отключающего) анализа оптимизаций компилятора. ACM Computing Surveys, Том 51, № 5, статья 96. Дата публикации: сентябрь 2018 года.

Таблица 7. Классификация, основанная на целевой платформе

Встроенная	Ссылки
платформа	[1, 3, 11, 15, 16, 21, 28, 29, 39, 41, 49, 50, 70, 78, 91, 92, 96, 101, 105, 135, 143, 148,
Домен	170, 171, 183-186, 195, 197, 202, 273]
Для рабочего стола	[3, 6, 10, 11, 18, 19, 28, 44, 51-55, 58, 61, 67, 69, 70, 79, 91, 96, 98-101, 105-108, 116, 122, 123, 135, 136, 142, 146, 151, 152, 155, 157, 162, 162, 163, 166, 169, 178, 183, 186, 187, 189, 196-202, 206, 208-210, 212, 213, 217, 219, 221, 230, 235-238, 240, 248, 250, 253, 259, 270, 271]
Домен НРС	[10, 11, 19, 36, 77, 83, 98, 101, 105, 109, 153, 159, 160, 175, 184, 198, 208, 209, 217, 221, 225-228, 240, 243, 249, 250, 270, 271]

Таблица 8. Классификация, основанная на целевом компиляторе

Компиляторы	Ссылки
ССАГПЗ	[3, 11, 15, 21, 28, 39, 40, 50, 61, 79, 83, 91, 92, 99, 101, 105, 107, 108, 112, 116, 122, 132, 135, 153, 155, 157, 160, 166, 169, 183, 189, 195-198, 200, 202, 205, 206, 208-210, 221, 230, 235, 240, 248, 253, 259, 270, 271]
LLVM	[14, 16, 18, 19, 21, 28, 29, 41, 72, 111, 112, 170, 171, 184-187, 199, 202, 212]
Intel-ICC	[44, 61, 91, 112, 135, 136, 160, 198, 200, 202, 209, 221, 240, 270]
JIT Компилятор	[52-55, 123, 126, 151, 152, 202, 213, 217, 221, 235]
Многогранный Компилятор	[52-55, 123, 126, 151, 213]
Java Модель	[43, 44, 199, 202, 208-210, 249, 258, 270, 271]
Прочее	[3, 6, 10, 11, 16, 28, 36, 51, 58, 67, 69, 70, 78, 91, 96, 98-101, 105, 106, 142, 146, 151, 152, 162, 162, 163, 170, 171, 178, 201, 202, 213, 216-219, 235, 236-238, 249, 250, 259]

7 ЦЕЛЕВОЙ ДОМЕН

Найти лучший набор оптимизаций компилятора, чтобы применить данное приложение является в значительной степени соответ- ся с типом компилятора, архитектуру целевого процессора и платформы должны быть настроены.

В исследованиях по автоматической настройке компилятора предпринимались попытки избежать универсальной оптимизации именно по этой причине. Используя модели машинного обучения, фреймворк должен быть адаптируемым к данному приложению или целевой платформе. Оптимизированный сегмент кода для данного компилятора или целевой платформы может не обеспечивать такой же оптимальности в другом компиляторе или платформе. У каждого компилятора и целевой комбинации платформ есть разные способы генерации двоичных файлов и выполнения сегментов кода . Более того, методы оптимизации могут быть полезны для определенного класса приложений, например, безопасности, научных и т.д., Но они могут не подходить для других классов. С этой целью мы классифицируем литературу на основе двух вышеупомянутых подклассов. Они показаны в таблицах. 7 и 8.

7.1 Целевая Платформа

Сегодня практически все программирование для настольных компьютеров и приложений НРС выполняется с использованием языков высокого уровня (как и большинство программ для встраиваемых приложений). Это наблюдение подразумевает, что поскольку большинство выполняемых инструкций являются выходными данными компилятора, архитектура набора инструкций - это

по сути, это цель компилятора [203]. Например, рассмотрим специальные инструкции цикла, найденные в приложении. Предположим, что вместо уменьшения на единицу компилятор хотел увеличить на четыре, или вместо разветвления при значении, не равном нулю, компилятор хотел разветвить, если индекс был меньше или равен пределу. В результате, цикл обучения может быть несоответствие имея отличаться- ЛОР целевой инструкция устанавливает MCA или архитектур. Выбор правильного набора оптимизаций с учетом архитектуры является необходимой задачей. Классификация, основанная на типе целевой платформы, показана в таблице 7.

Встроенный домен. С точки зрения встроенных вычислений, embedded охватывает почти все 7.1.1 вычисления, которые не считаются универсальными (GP) и высокопроизводительными вычислениями (HPC). Встроенные процессоры включают в себя большое количество интересных микросхем: тех, что используются в автомобилях, мобильных телефонах, пейджерах, портативных консолях, бытовой технике и другой бытовой электронике [90]. Некоторые из наиболее заметных встроенных архитектур представляют собой очень длинные слова инструкции (VLIW) [87, 88] и большой. НЕБОЛЬШАЯ гетерогенная архитектура от ARM [129]. В последнее время было много недорогих Проведенной платы с различной системой-на-чипе (SoC) характеристики, такие как малина Pi [252] и Pandaboard от Texas Instrument [125]. Читатели могут ознакомиться с уже доступными обзорами в области встраиваемых вычислений и ПЛИС [12, 65, 118]. Одним из ключевых отличий использования методов оптимизации компилятора для встраиваемой области является компромисс между размером кода приложения, производительностью и энергопотреблением. Оптимизация размера кода, особенно в архитектуре VLIW, была тщательно исследована [84, 163]. Однако из-за недавних достижений в размер кода встроенной SoC больше не является главной проблемой. Таким образом, фокус сместился в сторону показателей производительности, мощности и энергопотребления по методу Парето [13, 192, 194]. Методы оптимизации компилятора также могут быть использованы для решения этой задачи [29, 87, 184].

Намолару и др. [183] предложил общий метод систематической генерации числовых характеристик из приложения. Авторы реализовали свой подход поверх GCC. Этот метод не накладывает никаких ограничений на то, как логически и алгебраически объединять семантические свойства связей в числовые признаки; следовательно, он статистически охватывает всю релевантную информацию, которая может быть собрана из приложения. Они использовали статические функции MilePost GCC и MiBench для оценки своего подхода на встраиваемом процессоре ARC 725D.

Настольные компьютеры и рабочие станции. Несмотря на то, что большая часть исследований была 7.1.2 сделано для настольных компьютеров и рабочих станций, в последнее время акцент сместился в сторону обоих концов архитектурного спектра, а именно встроенных и высокопроизводительных вычислений. В этом разделе мы классифицировали эти работы по их экспериментальной установке, где целевой платформой не был ни один из двух вариантов, таким образом, мы классифицируем их как настольные компьютеры и категорию рабочих станций. Эта классификация приведена в таблице 7.

Томсон и др. [248] предложил метод кластеризации, позволяющий сократить время автономного обучения, присущее методам машинного обучения с обычным контролем. Авторы справились с этой задачей только на тех приложениях, которые лучше всего характеризуют оптимизации пространства. Они использовали алгоритм Густафсона-Кесселя после применения процесса уменьшения размеров и подход к кластеризации с помощью набора тестов EEMBCv2 и компьютера Intel Core 2 Duo E6750. Они экспериментально показали, что использование технологии может значительно сократить время

Область HPC. Существуют фундаментальные различия между кластером и суперкомпьютером. 7.1.3

Например, суперкомпьютеров и кластеров запускать несколько программ одновременно и поддерживает множество пользователей одновременно против суперкомпьютеров, которые направлены на вычислительной мощности для выполнения программы или указания как можно быстрее и ускорения производительности, чтобы подтолкнуть границы, что аппаратное и программное обеспечение может выполнить [47, 243]. Однако в целях краткости в этом

в обзоре мы поместили недавние работы, в которых использовались мэйнфреймы и кластеры, вместе с теми, в которых суперкомпьютеры использовались в качестве экспериментальной установки.

Тивари и др. [249] предложил масштабируемый фреймворк автоматической настройки, который включал серверную часть параллельного поиска Active Harmony [242] в фреймворке компилятора CHILL [59]. Предложенная методология позволила авторам параллельно исследовать пространство поиска и быстро находить улучшенные преобразованные версии ядра, которые обеспечивают более высокий прирост производительности.

Динг и др. [77] представил фреймворк автоматической настройки, способный использовать различные входные данные в двухуровневом подходе. Фреймворк построен на языке Petabricks и его компиляторе. [10]. Он использует технику обучения с учетом входных данных, чтобы различать входные данные, группировать пространство и выбирать его центроид для автоматической настройки. Двухуровневый подход состоит в определении набора конфигураций для каждого класса входных данных и создании классификатора для эффективного определения наилучшей оптимизации для использования для новых входных данных.

Фанг и др. [83] предлагают ИОД; итеративный подход к оптимизации, который мог бы быть развернут в центрах обработки данных. Они использовали внутренние характеристики центров обработки данных, чтобы преодолеть хорошо известное препятствие запуска метода итеративной компиляции, поскольку этот метод обычно требует большого количества запусков для каждого сценария. И подходит к решению этой задачи, создавая большое количество итеративных заданий компиляции одновременно для многих рабочих элементов и передавая их производительность обратно на главный узел, чтобы можно было быстро найти оптимальную политику компиляции. Более того, они оценивают свой подход с использованием MapReduce и подхода, требующего больших вычислительных ресурсов, например, `aws-om` флаги. Они использовали компилятор clang, предназначенный для многоядерного процессора ARM на плате ODROID и двух настольных компьютеров x86, представляющих узел в суперкомпьютерном центре.

7.2 Целевой компилятор

В параллелизме на уровне инструкций (ILP) и суперскалярных архитектурах [257], параллелизм в основном обусловлен эвристикой компилятора. Это понимание является ключевым для достижения максимальной производительности, энергоэффективности и снижения энергопотребления. Следовательно, важно направлять компилятор на поиск компромисса, который удовлетворяет конкретным целям платформы. "Типичные инвестиции в серверную часть компилятора до наступления зрелости измеряются в человеко-десятилетиях, и часто можно встретить платформы для компиляции с инвестициями в человеко-столетия" [87, 89]. Таблица 8 классифицирует последнюю литературу на основе типа используемой платформы компилятора.

7.2.1

GCC. GNU compiler collection (GCC) - это проект GNU compiler и toolchain, который

поддерживает различные языки высокого уровня, например, C, C++ и так далее. "Фонд свободного программного обеспечения (FSF) распространяет GCC под Общей общественной лицензией GNU (GNU GPL). GCC сыграл важную роль

в развитии свободного программного обеспечения, являясь одновременно инструментом и примером". [232, 233]. Этот проект был портирован на различные процессорные архитектуры, включая большинство встраиваемых систем (ARM, AMCC и Freescale), и может использоваться на многих целевых платформах. Из-за широкой поддержки и природы GCC с открытым исходным кодом, он был в центре внимания исследователей методов автоматической настройки компиляторов

. Стоит отметить, что GCC "из коробки" не поддерживает настройку с фазами его внутренних проходов компилятора, поскольку его диспетчер проходов переопределяет предопределенную упорядоченность. Однако изменение диспетчера проходов позволяет решить такую проблему оптимизации компилятора. Оптимизатор GCC⁵

поддерживает различные предопределенные уровни фиксированной оптимизации, такие как стандартные уровни `-O1`, `-O2` и `-O3`.

7.2.2

LLVM. Проект LLVM представляет собой набор модульных и повторно используемых компиляторов и инструментальных

цепочек технологий, используемых для разработки интерфейсов и серверных частей компилятора. Латнер и Викрам [154] определил описал LLVM как "фреймворк компилятора, предназначенный для поддержки прозрачной программы на протяжении всего срока

службы <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>.⁵

анализ и преобразование для произвольных программ путем предоставления высокоуровневой информации для преобразования компилятора во время компиляции, компоновки, выполнения и во время простоя между запусками". LLVM предлагает множество интересных функций: (i) Он имеет обычное промежуточное представление кода под названием *Intermediate Representation* статического одиночного присвоения (SSA). (ii) Его интерфейсная система на языке Си называется *LLVM IR* и предлагает множество языков расширений на основе pragmas. (iii) Его серверная часть была портирована на различные архитектуры, такие как x86-64, ARM, FPGA и даже графические процессоры. В последнее время сообщество LLVM превратилось в активное исследовательское сообщество, занимающееся портированием и внедрением новых функций в различные подмодули LLVM, например, (оптимизация), *clang* (Интерфейс языковой семьи C) и *lto* (обобщитель кода). Существует также множество исследовательских работ, связанных с использованием и построением LLVM.⁶

Intel-ICC. Компилятор *propriety* от Intel (ICC)⁷ обеспечивает общую оптимизацию, например, -o1,

7.2.3

-o2, -o3 и оптимизации для конкретного процессора в зависимости от целевой платформы. Более того, межпроцедурная оптимизация (IPO) - это автоматический многоступенчатый процесс, который позволяет компилятору анализировать ваш код, чтобы определить, где вы можете извлечь выгоду из конкретной оптимизации. С опциями IPO вы можете увидеть дополнительные оптимизации для микропроцессоров Intel по сравнению с микропроцессорами, отличными от Intel.

Компилятор "Точно в срок". Своевременная компиляция (JIT), также известная как динамический перевод, 7.2.4

это широко известная методика, которая используется уже много десятилетий. "В широком смысле, JIT-компиляция включает в себя любой перевод, выполняемый динамически после начала выполнения программы" [31]. Высокоуровневые преимущества использования JIT-компилятора можно резюмировать следующим образом: (i) Скомпилированные программы выполняются быстрее, особенно если они скомпилированы в форму, непосредственно исполняемую на базовом оборудовании. (ii) Интерпретируемые программы, как правило, более переносимы. (iii) Интерпретируемые программы могут получать доступ к информации о времени выполнения. Существует множество реализаций JIT-компиляторов, ориентированных на разные языки программирования. *Majic*, JIT-компилятор *Matlab* [7], *OpenJIT* [188], JIT-компилятор *Java* и JIT-компилятор *IBM* нацелены на виртуальную машину *Java* [241].

Компилятор *Java* специально разработан для компиляции *Java high-level pro-* 7.2.5

Компилятор Java.

код граммирования для генерации независимых от платформы классов байт-кода *Java*. Впоследствии Ява Виртуальной машины (JVM) могут загружать эти файлы и (I) выделять систему JIT, или (II) интерпретировать байт-код в целевой платформы машинный код. Оптимизация может быть выполнена в процессе, формально достигнутой их под удар компиляцию [2, 131]. Некоторые из известных исследовательских работ [52, 123], включая работу по решению проблемы упорядочения фаз, были выполнены с использованием *Java JIT compiler* [151]. Мы уже упоминали об этом в Разделе 4.2.2.

Многогранная модель. Сообщество по многогранной компиляции недавно привлекло внимание благодаря 7.2.6

к его широкому применению в проектах высокопроизводительных вычислений.⁸ "Многогранная компиляция объединяет методы компиляции, основанные на представлении программ, особенно тех, которые включают вложенные циклы и массивы, благодаря параметрическим многогранникам или отношениям Пресбургера" [85, 264]. Широкий спектр вычислительных и прикладных научных исследований проводят большую часть своего времени выполнения в цикле-гнезда, а этот подход для наведения на цель высокого уровня, компилятор оптимизаций [255]. Многогранная компиляция может решить многие из этих проблем путем эффективного преобразования их вложенного цикла [37, 44, 45, 161, 209]. Мы воздерживаемся от дальнейшего рассмотрения этой интересной темы, поскольку она выходит за рамки данного обзора.

Другие составители. В этом обзоре мы сосредоточились на классификации более широко 7.2.7

известный фреймворк компилятора в области автонастройки. Однако существует множество других хорошо

известных <http://llvm.org/pubs/>⁶

<https://software.intel.com/en-us/intel-компиляторы>.⁷

<http://polyhedral.info/>⁸

стоит упомянуть компиляторы, в том числе Cosy [9] и СУИФ [266]. Мы классифицировали всю работу, связанную с использованием другого компилятора, в таблице 8 в подполе "Другие".

8 САМЫХ ВЛИЯТЕЛЬНЫХ СТАТЕЙ

Оценка исследовательской работы - непростая задача, часто связанная с человеческой ошибкой. Однако в этом обзоре мы оцениваем и представляем влиятельные работы, используя их научный прорыв, новизну и конкурентный показатель цитирования. Мы дискретизировали процесс, представив влиятельные статьи по соответствующей теме и подробнее остановившись на предлагаемом ими подходе. Некоторые из них повлияли на их последующую работу, и это также было принято во внимание.

8.1 Достижения по темам

В следующих параграфах освещаются новые исследования в областях первоначального внедрения методов обучения с оптимизацией компилятора, генетических алгоритмов, упорядочения по фазам, итеративной компиляции, динамических и гибридных функций, создания групп оптимизации с байесовскими учащими и кластеризации оптимизаций для решения проблемы упорядочения по фазам.

Знакомство с методами обучения. Леверетт и др. и Вегдал и др. [156, 254] были первыми, кто сформировал неитеративную оптимизацию без использования машинного обучения. Ссылка [262] расширил эту работу, предложив разумный порядок подмножества оптимизаций компилятора. Ссылка [263] продолжила свои фазы-заказ работе с формального языка, Евангелие, которое может быть использовано для автоматически генерировать преобразований. Первое использование методов машинного обучения появилось со Ссылкой на епсе [143] и их работа по прогнозированию оптимального размера развертывания для вложенных циклов. Ссылка [180] был первым, кто использовал методы машинного обучения для построения гибких графиков обучения, прокладывая путь для продолжения усилий с использованием методов ML.

Генетические алгоритмы. Cooper et al. [69] расширенного машинного обучения, усилия по оптимизации лекция с использованием генетических алгоритмов с итерационными компиляции. Ссылка [70] расширили свою предыдущую работу, перейдя на адаптивную компиляцию и одну из самых ранних работ, создающую адаптивный фреймворк компиляции. Прогностическое моделирование было впервые введено по Ссылке [251] где они применили итеративную компиляцию тестов спецификации. Ссылка [142] предложена итеративная компиляция для выбора коэффициентов разбиения на плитки и развертывания с использованием генетических алгоритмов, случайной выборки и имитированного отжига..... "....." Они смогли показать, что их метод работает на многих различных архитектурах.

Упорядочение фаз. Кулкарни и др. [146] был одним из первых, кто предложил решить проблему упорядочения фаз с помощью машинного обучения путем сочетания итеративной компиляции и метаэвристики. Ссылка-епсе [151] решил проблему упорядочения фаз в виртуальной машине Java JIKES. Они используют устаревшие статические функции, загружаемые в нейронную сеть, сгенерированную с помощью NEAT, для построения хороших оптимизационных порядков последовательности.

Итеративная компиляция. Бодин и др. [42] предложил интеллектуальный метод итеративной компиляции, который исследовал менее 2% общего пространства в пространстве нелинейного поиска. Ссылка [3] использовал марковские цепочки для фокусировки итеративной оптимизации с использованием статических функций. Используя относительно небольшое исчерпывающее пространство поиска для обучения (14^5) и большое тестовое пространство для тестирования (80^{20}), они смогли добиться ускорения до 40%.

Динамические и гибридные функции. Первое использование динамических функций для обучения было введено Кавазосом и др. [51]; они показали, что использование динамических функций для обучения превосходит использование статических функций. Развитие многомерного (статического, динамического, гибридного) выбора характеристик и алгоритмов обучения проложило путь для предикторов турниров, представленных по ссылке [201].

Практическая автонастройка для совместной работы. MILEPOST GCC [101, 105] была первой попыткой создать практичный компилятор на основе машинного обучения "на лету" в сочетании с инфраструктурой, ориентированной

к сценариям автоматической настройки и краудсорсинга. Он использовался на практике и выявил множество проблем, которые еще предстоит решить исследователям, включая (1) воспроизводимость эмпирических результатов, собранных несколькими пользователями, и (2) проблемы с метаданными, представлением данных, моделями и массивными наборами данных [102, 104].

Гибридная характеристика и байесовские учащиеся. С помощью References был выполнен масштабный анализ набора данных по более чем 1000 контрольным отметкам [60, 61]. Они предложили группы последовательностей оптимизации (за пределами базовой линии традиционных компиляторов), которые в среднем выгодно использовать в приложениях из их набора данных. Совсем недавно Ашури и др. [15, 21] использовали выходные данные прохождений, предложенных [60] для построения байесовской сети для определения наилучшего флаги компилятора для конкретной области применения с нами-Инг статические, динамические и гибридные характеристики. Байесовская сеть генерировала последовательности оптимизации, в результате производительность приложений превзошла существующие модели.

Кластеризация оптимизации и предсказатели полной последовательности. Ашури и др. [19] представил MiCOMP фреймворк для кластеризации всех LLVM - оптимизация переходит в подпоследовательности оптимизации и представляем первый предиктор ускорения полной последовательности для задачи упорядочения фаз. Авторы использовали рекомендательную систему [214] подход к определенной политике исследования с использованием динамической информации, хранящейся в парных оптимизациях во всех обучающих приложениях, чтобы превзойти современный подход к ранжированию. Они показывают, что MiCOMP может достичь более 90% доступного ускорения и превосходства - оптимизация всего несколько прогнозов.

8.2 Достижения по производительности

В этом разделе приводится краткое количественное сравнение предлагаемых подходов относительно заявленных результатов. Оценка работ с учетом их эффективности является сложной задачей и часто сопряжена с ошибками сравнения [120], т. е. Использование абсолютных и относительных значений ускорения, усреднение с использованием различных методов и так далее. С этой целью мы изучаем документы, в которых явно представлено сравнение их предлагаемого подхода с их базовыми данными или современными методами.

Итеративная компиляция. Случайная итеративная компиляция (RIC), как известно, обеспечивает хорошее повторное- результаты при компиляции длительно работающих приложений [42]. Однако такой подход является дорогостоящим и должен сочетаться с интеллектуальными алгоритмами поиска, такими как методы машинного обучения [3, 19, 21, 42]. Ранние работы по методам итеративной компиляции [1, 42] включал исследование пространств нелинейных преобразований и нахождение максимально быстрого времени выполнения при фиксированном количестве оценок, ориентированных на встроенную область. Во встроенных доменах, стоимость выполнения последовательной модели compilation амортизируется по поводу размещения большого количества устройств. Исследователи использовали профильную информацию в виде времени выполнения, выполняли поиск по большому, но ограниченному подмножеству преобразований, чтобы найти хорошие результаты, и часто добивались значительного ускорения, исследуя лишь небольшую часть пространства оптимизации. Другие разработки превзошли эти подходы, внедрив более эффективные алгоритмы поиска в пространстве в дополнение к рассмотрению более обширного пространства оптимизации с различными коэффициентами развертывания и размерами плиток [142]. Мпейс и др. [181] реализован итеративный подход к сборке, ориентированный на мобильные устройства, для дальнейшей оптимизации байт-кода Java повседневного запущенного приложения, когда устройство не использовалось. Авторы наблюдали в среднем улучшение производительности на 57% при использовании бенчмарка с минимальными замедлениями. В других крупных работах делались попытки объединить методологию итеративной компиляции с методами машинного обучения, и мы обсудим эту категорию далее.

8.2.2 Методы машинного обучения.

Проблема выбора Агаветал.[3] представил подход машинного обучения, позволяющий сосредоточиться на итеративной компиляции. Их подход с использованием статических функций и марковского Oracle привел к 22% и 27% ACM Computing Surveys, Том 51, № 5, статья 96. Дата публикации: сентябрь 2018 г.

повышение производительности Texas instrument и архитектуры AMD. Позже Кавазос и др.

[51] превзошел предыдущую работу на 7% с использованием метода динамической характеристики объекта.

Авторы полагали, что использование статических функций приложения, хотя и достаточно хорошо для встроенных мультимедийных приложений, но не может хорошо работать в крупномасштабных приложениях общего назначения.

Использование подходов прогностического моделирования было продолжено другими авторами [97, 105, 122]. Паркетная

доска. [201 позже] внедрил новый подход с использованием предсказателей турниров (см. Раздел 5.4), благодаря чему

они превосходили существующие методы примерно на 6%. Далее авторы предложили использовать ядра graph

[199] (см. Раздел 5.4) как средство улучшения существующих моделей прогнозирования. Ашури и др.

представил КОБЕЙН [21], в котором использовались гибридные функции и средства обучения байесовской сети. Авторы

показали, что такой подход может превзойти существующие методы [3, 202] примерно на 11%.

Проблема упорядочения фаз. Медь. [70] предложил механизм поиска для нахождения правильного

упорядочения фаз с использованием генетического алгоритма. Подход основывался на адаптивной компиляции

потока, который мог ускорить до 49% в данном тесте. Позже Кулкарни и др. [146] предложил

еще один генетический алгоритм, который смог резко сократить время поиска до

65%. Первый промежуточный предиктор ускорения был позже предложен Кулкарни и Кавазосом [151].

Авторы использовали характеристику текущего состояния кода и NEAT (см. раздел

4.2.2) и получили ускорение до 20% в приложениях, использующих компилятор Java Jikes. Ашури и др.

предложили MICOMP [19], который использует подпоследовательности, полученные из LLVM - O3 и предиктор полной

последовательности ускорения. Авторы показали, что, используя свой подход, они могут превзойти существующие

подходы [18, 151] на 5% и 11% соответственно.

8.2.3 *Многоцелевая оптимизация.* В этом подразделе мы кратко обсудим количественные результаты

результатов исследования основных целей оптимизации, описанных в литературе. Этими целями

являются размер кода, площадь, показатели мощности и производительности.

Размер и площадь кода. Ранние работы были нацелены на уменьшение размера кода в качестве цели оптимизации.

Купер и др. [69] представил метод генетического алгоритма (GA) для поиска уменьшенного размера кода и обнаружил,

что он имеет преимущество в 14,5% по сравнению с фиксированной последовательностью по умолчанию. За этими работами

последовал ряд других, в основном связанных с уменьшением размера кода во встроенной архитектуре VLIW [13,

29, 90, 268]. Однако благодаря развитию систем хранения данных, особенно во встроенной

области, этот вопрос стал вызывать гораздо меньше беспокойства и часто игнорировался в недавней

литературе.

Производительность и интенсивность. Модель линии крыши [265] относится к производительности процессора на чип мем-ори движения, обеспечивает, таким образом, теоретическая верхняя граница эксплуатационной интенсивности и достигаемого

перфор-Манс в современных архитектур. В некоторых работах это понятие использовалось для моделирования их

оптимизационных подходов и нахождения кривой Парето, удовлетворяющей обоим целям [13, 16, 29, 75, 122, 194, 272]. Хосте

и Экхаут [122] взяли за итеративную компиляцию на компьютере Intel Pentium и наблюдали за

осуществимостью их подхода вместе с многоцелевым алгоритмом поиска, называемым множественными заполнениями,

ulations, который находит кривые Парето в пределах различной совокупности пространства оптимизации. Они

наблюдали ускорение до 37%, когда использовали свою оценочную метрику. Ашури и др. [29] применил

модель линии крыши к индивидуальной архитектуре VLIW и сформировал четыре кластера для удовлетворения многоцелевого

предложенного сценария. Они отметили ускорение времени выполнения до 23% за счет

использования тех оптимизаций компилятора, которые были найдены в их методе и которые способствовали достижению

поставленных целей.

Мощность За исключением нескольких недавних работ [184, 258], эта цель была в основном исследована

на архитектурном и системном уровнях [13, 30, 158, 194]. По этой причине это выходит за рамки данного

исследования.

Таблица 9. Классификация 16 самых влиятельных статей по количеству цитирований

№	Ссылка	Количество цитирований	АСРУ	Ключевые слова
1	Агаков и др. [2006] [13]	377	33	итеративная компиляция, статические функции, опракул цепи
2	Купер и др. [1999] [69]	322	17	Маркова, PCA генетический алгоритм, итеративная компиляция.
3	Тристафиллис и др. [2003] [764]	277	19	уменьшенный размер кода, итеративная компиляция,
4	Стивенсон и др. [2003] [236]	274	19	СПЕЦИФИКАЦИЯ, прогнозирующее моделирование,
5	Фурсин и др. [2008, 2011] [101, 105]	251	22	итеративная компиляция, генетическое программирование, метазвистика, промежуточный GCC, самонастройка,
6	Купер и др. [2002] [70]	242	15	краудсорсинг, Итеративная компиляция, предиктор последовательности, итеративная компиляция, фактор
7	Klijnburg et al. [2003] [142]	238	16	развертывания, независимый от архитектуры статический анализ, параллелизм, управляемый профилем, NAS, СПЕЦИФИКАЦИЯ
8	Турнавитис и др. [2009] [250]	234	27	итеративная компиляция, динамическая характеристика,
9	Кавазос и др. [2007] [51]	225	20	предиктор последовательности, фреймворк CHILL framework, итеративная компиляция, предиктор последовательности,
10	Тивари и др. [2009] [140]	200	25	адаптивная компиляция, предиктор последовательности
11	Альмагос и др. [2004] [61]	196	14	компилятора, SPARC деревья решений, повышение, представление
12	Монсифро и др. [2002] [178]	175	11	абстрактного цикла, контролируемое обучение, фактор развертывания,
13	Стивенсон и др. [2005] [237]	170	13	многоклассовая классификация, комбинированное
14	Пан и др. [2006] [197]	161	13	исключение, итеративная компиляция, СПЕЦИФИКАЦИЯ,
15	Бодин и др. [1998] [40]	151	7	итеративная компиляция, многоцелевое исследование,
16	Hoste et al. [2008] [122]	134	13	итеративная компиляция, метазвистика, генетический алгоритм

8.3 Показатель цитирования

В этом разделе мы показываем 16 наиболее цитируемых статей из более чем 200 подготовленных нами.⁹ В таблице 9, мы представляем их количество цитирований и среднее значение цитируемости за год (АСРУ) с несколькими ключевыми словами, представляющими их методологию, которую мы уже рассматривали в этом опросе.

9 ОБСУЖДЕНИЕ И ЗАКЛЮЧЕНИЕ

В ближайшие десятилетия исследования методов компиляции и оптимизации кода будут играть ключевую роль в решении различных проблем информатики и области высокопроизводительных вычислений. Это включает автоматическое распараллеливание, безопасность, использование многоядерных процессоров, надежность, воспроизводимость и энергоэффективность. Использование оптимизаций компилятора для использования крупномасштабного параллелизма, доступного в архитектурах и на мощном оборудовании, является важной задачей. Кроме того, машинное обучение становится все более эффективным благодаря использованию глубокого обучения для поиска и построения эвристик. Эти сложные обучающие программы позволяют автоматизированным системам эффективно выполнять эти задачи с минимальными усилиями программиста.

Кроме того, исследования по методологиям совместной настройки привлекли внимание благодаря внедрению системы коллективных знаний (СК). [101-103, 174]. СК - это кроссплатформенный открытый исследовательский SDK, разработанный в сотрудничестве с академическими и промышленными партнерами для обмена артефактами в виде многоразовых и настраиваемых компонентов с унифицированными, переносимыми и настраиваемыми рабочими процессами для экспериментов . Исследователи могут автоматизировать процесс настройки различного оборудования и сред . Также возможен краудсорсинг и воспроизведение экспериментов на платформах, предоставляемых другими поисковиками.

⁹СК теперь используется в качестве официальной платформы для поддержки open ACM .
Данные были извлечены из Google Scholar в июле 2018 года и могут быть изменены.

Организуем турниры по воспроизводимому и эффективному по Парето программному и аппаратному обеспечению для совместного проектирования искусственного интеллекта, глубокого обучения и других новых рабочих нагрузок. [- 179].

С появлением *Глубокого обучения*, то есть глубоких нейронных сетей (DNN) [222], мы являемся свидетелями все большего и большего применения таких методов, т.е. распознавания речи [119], классификация изображений [144], и так далее. Алгоритмы глубокого обучения извлекают высокоуровневые и сложные абстракции из данных посредством иерархического процесса обучения и показали эффективность во многих приложениях. DNN обычно требуют больших объемов данных для обучения, поэтому адаптация новых контрольных показателей и больших наборов данных сыграет важную роль в достижении потенциальных преимуществ использования DNN. Недавно мы видели эталонные синтезаторы, которые способны генерировать синтетические программы для использования в автоматической настройке производительности на основе машинного обучения. Бытие [63] и CLgen [74] являются такими примерами, которые привносят разнообразие и контроль в процесс создания новых тестов для пользователя.

В этом обзоре мы обобщили исследовательскую работу по автоматической настройке компилятора с использованием машинного обучения, показав широкий спектр использования методов машинного обучения и их ключевые исследовательские идеи и приложения. Опрошенные нами научно-исследовательские работы на разных уровнях абстракции, API-пакеты методов исследования, алгоритмы и модели машинного обучения, прогнозирования типов, освоение космоса, Домены назначения и влияния. Мы обсудили обе основные проблемы автоматической настройки компилятора, а именно проблему выбора и упорядочения фаз, а также бенчмарки, предложенные для их оценки. Мы надеемся, что эта статья будет полезна компьютерным архитекторам, исследователям и разработчикам приложений, вдохновит на новые идеи и откроет многообещающие направления исследований.

СПИСОК ЛИТЕРАТУРЫ

- [1] Бас Артс, Мишель Баррето, Франсуа Боден, Питер Бринкхаус, Збигнев Чамски, Анри-Пьер Шарль, Кристиан Эйзенбейс, Джон Герд, Ян Хугербрюгге, Пинг Ху и др. 1997. ОКЕАНЫ: оптимизация компиляторов для встраиваемых приложений. катюны. В *Материалы Европейской конференции по параллельной обработке данных (Euro-Par '97)*. 1351-1356. [2] Али-Реза Адль-Табатабаи, Михал Черняк, Гуэй-Юань Луэ, Вишеш М. Парих и Джеймс М. Стихот. 1998. Быстрая, эффективная генерация кода в компиляторе Java "точно в срок". В *Примечаниях ACM SIGPLAN*, Том 33. ACM, 280-290.
- [3] Феликс Агаков, Эдвин Бонилья, Джон Кавазос, Бьерн Франке, Григорий Фурсин, Майкл Ф. П. О'Бойл, Джон Томсон, Марк Туссен и Кристофер К. И. Уильямс. 2006. Использование машинного обучения для итеративной оптимизации. В *Материалы Международного симпозиума по генерации и оптимизации кода*. IEEE, 295-305. [4] Альфред В. Ахо, Рави Сети и Джеффри Д. Ульман. 1986. *Компиляторы, принципы, методы..* Эддисон Уэсли. [5] Frances E. Allen. 1970. Анализ потока управления. В разделе *Примечания ACM Sigplan*, Том 5. ACM, 1-19. [6] Л. Альмагор и К. Д. Купер. 2004. Поиск эффективных последовательностей компиляции. *Примечания к ACM SIGPLAN* 39, 7 (2004), 231-239. Извлечено из <http://www.anc.ed.ac.uk/machine-learning/colo/repository/LCTES04.pdf>.
- [7] Джордж Алмази и Дэвид А. Падуа. 2000. MaJIC: компилятор MATLAB "точно в срок". В *Трудах Международной международной семинар по языкам и компиляторам для параллельных вычислений*. Спрингер, 68-81.
- [8] Ethem Alpaydin. 2014. *Введение в машинное обучение*. MIT Press. [9] Martin Alt, Uwe Altmann, and Hans Van Someren. 1994. Внедрение фазы компиляции Cosy с помощью компилятора cosy Модель. В *Материалы Международной конференции по конструированию компиляторов* Спрингер, 278-293.
- [10] Джейсон Ансел, Сай Чан, Йи Лок Вонг, Марек Ольшевски, Цинь Чжао, Алан Эдельман и Саман Амарасинке. 2009. PetaBricks: язык и компилятор для алгоритмического выбора. В *Материалах 30-й конференции ACM SIGPLAN по проектированию и реализации языков программирования (PLDI'09)* ACM, Нью-Йорк, Нью-Йорк, 38-49. doi: <http://dx.doi.org/10.1145/1542476.1542481>
- [11] Дж. Ансель и С. Камилл. В разделе *Материалы 23-й Международной конференции по параллельным архитектурам и компиляции*. 303-316.
- [12] Карл-Эрик Орзен и Антон Червин. 2005. Контроль и встроенные вычисления: Обзор направлений исследований. *Сборник трудов*, 38, 1 (2005), 191-202.
- [13] Г. Ассия, В. Катания, М. Палези и Д. Патти. 2005. Структура системного уровня для оценки площади / производительности / мощности Компромиссы встроенных систем на основе VLIW. В разделе *Материалы конференции по автоматизации проектирования (ASP-DAC'05)*, Том 2. 940-943.
- [14] Йоси Бен Ашер, Гади Хабер и Эсти Штайн. 2017. Исследование конфликтующих пар оптимизаций компилятора. В *Продолжение-доклады 11-го Международного симпозиума IEEE по встраиваемым многоядерным системам на кристалле (MCSoc'17)*. IEEE, 52-58.

IFAC

- [15] А. Х. Ашури, Г. Мариани, Г. Палермо и К. Сильвано. 2014. Байесовский сетевой подход к автоматической настройке компилятора для встроенных процессоров. В *Материалы конференции IEEE Embedded Systems for Real-Time Multimedia (ESTIMedia)*. 90-97. doi : <http://dx.doi.org/10.1109/ESTIMedia.2014.6962349>
- [16] Амир Хоссейн Ашури. 2012. *Разработка методологии исследования пространства для параметров компилятора в процессорах VLIW.Mas-* докторская диссертация тех. Politecnico Di Milano, Italy. Извлечено из <http://hdl.handle.net/10589/72083>. [17] Амир Хоссейн Ашури. 2016. *Автонастройка компилятора с использованием методов машинного обучения*. Докторская диссертация. Политехнический- nico di Milano, Italy. Извлечено из <http://hdl.handle.net/10589/129561>.
- [18] Амира Хоссейна Ашури, Андреа Биньоли, Джанлука Палермо и Кристины Сильвано. 2016. Метод прогностического моделирования.- логика для упорядочивания этапов компилятора. В *Труды 7-го семинара по параллельным вычислениям и времени выполнения человеком- нежментя приемы для многоядерных архитектур и 5-м заседаниях рабочей группы по разработке инструментов и архитектур для мульти- ядра встраиваемых вычислительных платформ (Парма-DITAM'16)* ACM, Нью-Йорк, Нью-Йорк, 7-12. doi : <http://dx.doi.org/10.1145/2872421.2872424>
- [19] Амир Х. Ашури, Андреа Биньоли, Джанлука Палермо, Кристина Сильвано, Самир Кулкарни и Джон Кавазос. 2017. MiCOMP: Решение проблемы упорядочения фаз компилятора с помощью подпоследовательностей оптимизации и машинного обучения. *ACM Trans. Archit. Optimizuriune kod*. Пункт 3 статьи 14, статья 29 (сентябрь 2017 г.). doi : <http://dx.doi.org/10.1145/3124452>
- [20] Амир Х. Ашури, Уильям Киллиан, Джон Кавазос, Джанлука Палермо и Кристина Сильвано. 2018. Обзор компилятора Автонастройка с использованием машинного обучения. *Препринт arXiv arXiv:1801.04405* (2018).
- [21] Амир Хоссейн Ашури, Джованни Мариани, Джанлука Палермо, Ынджунг Парк, Джон Кавазос и Кристина Сильвано. 2016. КОБЕЙН: Фреймворк автоматической настройки компилятора с использованием байесовских сетей. *ACM Trans. Archit. Code Optim.* 13, 2, Статья 21 (июнь 2016 г.). doi : <http://dx.doi.org/10.1145/2928270>
- [22] Амир Х. Ашури, Джанлука Палермо, Джон Кавазос и Кристина Сильвано. 2018. *Автоматическая настройка компиляторов с использованием Машинного обучения*. Издательство Springer International Publishing. <http://dx.doi.org/10.1007/978-3-319-71489-9> doi : <http://dx.doi.org/10.1007/978-3-319-71489-9>
- [23] Амир Х. Ашури, Джанлука Палермо, Джон Кавазос и Кристина Сильвано. 2018. *Справочная информация*. Springer International Publishing, Cham, 1-22. http://dx.doi.org/10.1007/978-3-319-71489-9_1 doi : http://dx.doi.org/10.1007/978-3-319-71489-9_1
- [24] Амир Х. Ашури, Джанлука Палермо, Джон Кавазос и Кристина Сильвано. 2018. *Проектное исследование пространства Com- Проходы пилера: подход к совместному исследованию встроенной области*. Издательство Springer International Publishing, Cham, 23-39. doi : http://dx.doi.org/10.1007/978-3-319-71489-9_2
- [25] Амир Х. Ашури, Джанлука Палермо, Джон Кавазос и Кристина Сильвано. 2018. *Проблема упорядочения фаз: комплексный полный подход к прогнозированию последовательности* Издательство Springer International Publishing, Cham, 85-113. doi : http://dx.doi.org/10.1007/978-3-319-71489-9_5
- [26] Амир Х. Ашури, Джанлука Палермо, Джон Кавазос и Кристина Сильвано. 2018. *Проблема упорядочения фаз: Подход к прогнозированию промежуточного ускорения*. Издательство Springer International Publishing, Cham, 71-83. doi : http://dx.doi.org/10.1007/978-3-319-71489-9_4
- [27] Н. Ashouri Амир, Джанлука Палермо, Джон Кавазос, и Кристина Сильвано. 2018. *Выбрав лучший компилятор оптимизации- ний: Байесовский сетевой подход* Издательство Springer International Publishing, Cham, 41-70. doi : http://dx.doi.org/10.1007/978-3-319-71489-9_3
- [28] Амир Хоссейн Ашури, Джанлука Палермо и Кристина Сильвано. Оценка методов автоматической настройки для *Материалы семинара по осведомленности о ресурсах и автоматической настройке приложений проблемы оптимизации компилятора. В области адаптивных и гетерогенных вычислений (RES4ANT'16), приуроченный к конференции и выставке "Автоматизация проектирования и тестирования в Европе" (ДАТА ПРОВЕДЕНИЯ'16)*. 23-27. <http://ceur-ws.org/Vol-1643/#paper-05>
- [29] Амир Хоссейн Ашури, Витторио Заккария, Сотирис Ксидис, Джанлука Палермо и Кристина Сильвано. 2013. Структура для статистического анализа на уровне компилятора с использованием настраиваемой архитектуры VLIW. В *Материалах Международной конференции по очень крупномасштабной интеграции (VLSI-SoC'13)*. 124-129. doi : <http://dx.doi.org/10.1109/VLSI-SoC.2013.6673262>
- [30] Хосе Л. Айала, Мариса Лопес-Вальехо, Дэвид Атьенса, Правин Рагхаван, Фрэнки Кэттур и Дидерик Веркест. 2007. Энергоэффективная компиляция и проектирование оборудования для встраиваемых систем VLIW. *Int. J. Встраивать. Syst.* 3, 1-2 (2007), 73-82.
- [31] Джон Эйкок. 2003. Краткая история just-in-time. *ACM Вычисляет. Обзоры* 35, 2 (2003), 97-113.
- [32] Р. Бабука, П. Дж. Ван дер Вин и У. Каймак. 2002. Улучшенная оценка ковариации для кластеризации Густафсона-Кесселя. В *Материалах Международной конференции IEEE 2002 года по нечетким системам (FUZZ-IEEE'02)*, том. 2. IEEE, 1081-1085.
- [33] Дэвид Ф. Бэкон, Сьюзан Л. Грэм и Оливер Дж. Шарп. 1994. Преобразования компилятора для высокопроизводительных вычислений- ввод. *Вычисления. Обзоры* 26, 4 (декабрь 1994 г.), 345-420. doi : <http://dx.doi.org/10.1145/197405.197406>
- [34] Виктор Р. Бэзил и Альберт Дж. Тернер. 1975. Итеративное улучшение: Практический метод разработки программного обеспечения. *Перевод IEEE. Softw. Eng.* 4 (1975), 390-396.
- [35] Протону Басу, Мэри Холл, Малик Хан, Сучит Майндоло, Саурав Муралидхаран, Шрейас Рамалингам, Аксель Ривера., Ману Шантарам и Ананд Венкат. 2013. К тому, чтобы сделать автонастройку мейнстримом. *Int. J. Высокая производительность. Вычисление. Приложение.* 27, 4 (2013), 379-393.

- [36] Протону Басу, Сэмюэл Уильямс, Брайан Ван Страален, Леонид Оликер, Филипп Колелла и Мэри Холл. 2017. Составитель-генерация кода на основе и автонастройка геометрической многосетки на суперкомпьютерах с графическим ускорением. *Параллельный вычислительный способ*. 64 (2017), 50-64.
- [37] Мохаммед-Валид Бенабдеррахман, Луи-Нозль Пуше, Альберт Козн и Седрик Бастул. 2010. Многогранная модель Применима шире, чем вы думаете. В разделе *Конструкция компилятора*. Спрингер, 283-303.
- [38] Крейг Блэкмор, Оливер Рэй и Керстин Эдер. 2015. Подход к логическому программированию для прогнозирования эффективности компилятора настройки встроенного программного обеспечения. *Теоретическая практика. Логическая программа*. 15, 4-5 (2015), 481-494.
- [39] Крейг Блэкмор, Оливер Рэй и Керстин Эдер. 2017. Автоматическая настройка компилятора GCC для оптимизации-формирование приложений, работающих на ARM cortex-M3. *Препринт arXiv arXiv:1703.08228* (2017).
- [40] Крейг Блэкмор, Оливер Рэй и Керстин Эдер. 2017. Автоматическая настройка компилятора GCC для оптимизации производительности-приложений, работающих на ARM cortex-M3. *CoPP*. abs/1703.08228 (2017). arXiv: 1703.08228, извлечено из <http://arxiv.org/abs/1703.08228>. [41] Бруно Бодин, Луиджи Нарди, М. Зишан Зиа, Гарри Вагстафф, Говинд Срикара Шеной, Мурали Эмани, Джон Мавер, Кристоф Коцелидис, Энди Нисбет, Микель Лухан и др. 2016. Интеграция алгоритмических параметров в бенчмаркинг и исследование проектного пространства для понимания 3D-сцен. В *Материалы Международной конференции 2016 года по параллельным архитектурам и компиляции*. ACM, 57-69.
- [42] Франсуа Боден, Торус Кисук, Питер Книненбург, Майк О'Бойл и Эрвен Роу. 1998. Итеративная компиляция в пространстве нелинейной оптимизации. В *Материалах семинара по компиляции с профилем и обратной связью*.
- [43] У. Бондхугула и М. Баскаран. 2008. Автоматические преобразования для распараллеливания с минимизацией связи и оптимизация локальности в многогранной модели. В *Труды Международной конференции по конструированию компиляторов*. 132-146. Извлечено из <http://link.springer.com/chapter/10.1007/978-3-540-78791-4>. [44] У. Бондхугула и А. Хартоно. 2008. Практичный автоматический многогранный распараллеливатель и оптимизатор локальности. (2008). Извлечено из <http://dl.acm.org/citation.cfm?id=1375595>.
- [45] Удай Бондхугула, А. Хартоно, Дж. Рамануджам и П. Садаппан. 2008. PLuTo: Практичный и полностью автоматический многогранная система оптимизации программ. В *Материалах конференции ACM SIGPLAN 2008 по программированию Языковой дизайн и реализация (PLDI'08)*. Citeseer. [46] Karsten M. Borgwardt and Hans-Peter Kriegel. 2005. Ядра кратчайшего пути на графах. В *Труды 5-го IEEE Международная конференция по интеллектуальному анализу данных (ICDM'05)*. IEEE, 8-стр.
- [47] Раджумар Буйя, Чи Шин Йео и Шрикумар Венугонал. 2008. Облачные вычисления, ориентированные на рынок: видение, шумиха, и реальность предоставления ИТ-услуг в виде вычислительных утилит. В *Материалах 10-й Международной конференции IEEE по высокопроизводительным вычислениям и коммуникациям (HPCC'08)*. IEEE, 5-13. [48] Густаво Кэмпис-Вальс, Татьяна В. Бандос Маршева и Денген Чжоу. 2007. Ну на основе полуправляемых графиков-классификация изображений в перспективе. *Перевод IEEE. Geosci. Дистанционный датчик*. 45, 10 (2007), 3044-3054.
- [49] João Manuel Paiva Cardoso, José Gabriel de Figueiredo Coutinho, and Pedro C. Diniz. 2017. Встроенные вычисления для Высокая производительность: Эффективное сопоставление вычислений с использованием кастомизации, преобразований кода и компиляции. Morgan Kaufmann. [50] Дж. Кавазос, К. Дубач и Ф. Агаков. 2006. Автоматическое построение модели производительности для быстрого развертывания программного обеспечения-анализ новых конструкций оборудования. В *Материалы Международной конференции 2006 года по компиляторам, архитектуре и Синтез для встраиваемых систем*. 24-34. Извлечено из <http://dl.acm.org/citation.cfm?id=1176765>.
- [51] Дж. Кавазос, Г. Фурсин и Ф. Агаков. 2007. Быстрый выбор хороших оптимизаций компилятора с использованием показателей производительности. *Материалы Международного симпозиума по генерации и оптимизации кода (CGO'07)*. Извлечено из <http://ieeexplore.ieee.org/xpls/abs>. [52] Дж. Кавазос и Дж. Э. Б. Мосс. 2004. Побуждение эвристики решать, следует ли планировать. *Уведомления ACM SIGPLAN* (2004). Извлечено из <http://dl.acm.org/citation.cfm?id=996864>.
- [53] Дж. Кавазос, Дж. Э. Б. Мосс и М. Ф. П. О'Бойл. 2006. Гибридные оптимизации: какой алгоритм оптимизации использовать? *Построение компилятора* (2006). Извлечено из <http://link.springer.com/chapter/10.1007/11688839>.
- [54] Дж. Кавазос и М. Ф. П. О'Бойл. 2005. Автоматическая настройка встроенных эвристик. *Материалы ACM/IEEE SC 2005 Конференция по суперкомпьютерам*. 14-14. Извлечено из <http://ieeexplore.ieee.org/xpls/abs>.
- [55] Дж. Кавазоса и М. Ф. П. О'Бойла. 2006. Динамическая компиляция для конкретного метода с использованием логистической регрессии. *ACM SIGPLAN Примечания* (2006). Извлечено из <http://dl.acm.org/citation.cfm?id=1167492>.
- [56] Грегори Дж. Чайтин, Марк А. Ауслендер, Ашок К. Чандра, Джон Кок, Мартин Э. Хопкинс и Питер В. Маркштейн. 1981. Выделение регистра с помощью раскраски. *Вычисление. Lang.* 6, 1 (1981), 47-57.
- [57] Оливье Шапель, Бернхард Шолкопф и Александр Зиен. 2009. Обучение под контролем (О. Шапель и др., ред.). *Перевод IEEE. Нейронная сеть*. 20, 3 (2009), 542-542.
- [58] К. Чен, Дж. Чейм и М. Холл. 2005. Объединение моделей и управляемый эмпирический поиск для оптимизации на нескольких уровнях иерархии памяти. *Материалы Международного симпозиума по генерации кода и оптимизации*. Извлечено из <http://ieeexplore.ieee.org/xpls/abs>.

- [59] Чун Чен, Жаклин Чейм и Мэри Холл. 2008. *Охлаждение: основа для составления высокоуровневых преобразований цикла-размышления*. Технический отчет. CiteSeer.
- [60] Ян Чен, Шуандз Фанг, Юаньцзе Хуан, Ливен Экхаут, Григорий Фурсин, Оливье Темам и Ченген Ву. 2012. Деконструкция итеративной оптимизации. *ACM Trans. Архитектор. Оптимизация кода*. 9, 3 (2012), 21.
- [61] Ян Чен, Юаньцзе Хуан, Ливен Экхаут, Григорий Фурсин, Лян Пэн, Оливье Темам и Ченген Ву. 2010. Оценка итеративной оптимизации для 1000 наборов данных. В *Материалах конференции ACM SIGPLAN 2010 года по проектированию и реализации языков программирования (PLDI'10)*. ACM, Нью-Йорк, Нью-Йорк, 448-459.
- doi: <http://dx.doi.org/10.1145/1806596.1806647>
- [62] Б. Р. Чайлдс и М. Л. Соффа. 2005. Фреймворк на основе моделей: подход к оптимизации, ориентированной на прибыль. В *Материалах Международного симпозиума по генерации и оптимизации кода*. IEEE, 317-327. doi: <http://dx.doi.org/10.1109/CGO.2005.2>
- [63] Элтон Чиу, Джозеф Гарви и Тарек С. Абдельрахман. 2015. Genesis: Язык для создания синтетического обучения программы для машинного обучения. В *Материалах 12-й Международной конференции ACM по компьютерным границам*. ACM, 8. [64] Клифф Клик и Кит Д. Купер. 1995. Комбинирование анализов, комбинирование оптимизаций. *ACM Trans. Программа. Lang. Система*. 17, 2 (1995), 181-196.
- [65] Кэтрин Комптон и Скотт Хаук. 2002. Реконфигурируемые вычисления: обзор систем и программного обеспечения. *ACM Вычисления. Обзоры* 34, 2 (2002), 171-210.
- [66] Кэтрин Э. Кунс, Бехнам Робатмили, Мэтью Э. Тейлор, Бертран А. Махер, Дуг Бергер и Кэтрин С. Маккин-лей. 2008. Выбор функций и оптимизация политики для распределенного размещения инструкций с использованием обучения с подкреплением. В *Материалах 17-й Международной конференции по параллельным архитектурам и методам компиляции*. ACM, 32-42.
- [67] К. Д. Купер, А. Гросул и Т. Дж. Харви. 2005. ACME: Адаптивная компиляция стала эффективной. В *ACM SIGPLAN Уведомления* 40, 7 (2005), 69-77. Извлечено из <http://dl.acm.org/citation.cfm?id=1065921>.
- [68] К. Купера, Тимоти Дж. Харви, Девки Субрамания и Линды Торчон. 2002. *Порядок компиляции имеет значение*. Технические Сообщить.
- [69] К. Д. Купер, П. Дж. Шильке и Д. Субрамания. 1999. Оптимизация для сокращения объема кода с использованием генетических алгоритмов. *Уведомления ACM SIGPLAN*. Извлечено из <http://dl.acm.org/citation.cfm?id=314414>.
- [70] К. Д. Купера, Д. Субрамания и Л. Торчона. 2002. Адаптивные оптимизирующие компиляторы для 21 века. *Дж. Супер-вычисление*. Извлечено из <http://link.springer.com/article/10.1023/A:1015729001611>.
- [71] Бьяджо Козенца, Хуан Дж. Дурильо, Стефано Эрмон и Бен Юурлинк. 2017. Автонастройка трафарета с порядковой регрессией.: Расширенный реферат. В *Материалах 20-го Международного семинара по программному обеспечению и компиляторам для встраиваемых систем (SCOPES'17)*. ACM, Нью-Йорк, NY, 72-75. doi: <http://dx.doi.org/10.1145/3078659.3078664>
- [72] Крис Камминс, Павлос Петуменос, Мишель Стювер и Хью Лезер. 2015. Автонастройка размера рабочей группы OpenCL для шаблонов трафаретов. *Препринт arXiv arXiv: 1511.02490*.
- [73] С. Камминс, П. Петуменос, З. Ван и Х. Кожан. 2017. Комплексное глубокое изучение эвристики оптимизации. В *Материалах 26-й Международной конференции по параллельным архитектурам и методам компиляции (PACT'17)*. 219-232. doi: <http://dx.doi.org/10.1109/PACT.2017.24>
- [74] Крис Камминс, Павлос Петуменос, Чжэн Ван и Хью Лезер. 2017. Синтез контрольных показателей для прогнозного моделирования. В *Материалах Международного симпозиума IEEE / ACM по генерации и оптимизации кода (CGO'17)*. IEEE, 86-99. [75] Каланмой Деб, Амрит Пратап, Самир Агарвал и Т. А. М. Т. Мехриван. 2002. Быстрый и элитарный мультиобъектный алгоритм генетический алгоритм: NSGA-II. *Перевод IEEE. Evol. Вычисление*. 6, 2 (2002), 182-197.
- [76] Томас Г. Диттерих. 2000. Ансамбль методов машинного обучения. В *Международный семинар по нескольким классификатор систем*. Спрингер, 1-15.
- [77] Ю. Динг, Дж. Ансель и К. Вирамачанени. 2015. Выбор алгоритма автоматической настройки чувствительности ввода. *ACM SIGPLAN Примечания* 50, 6 (2015), 379-390. Извлечено из <http://dl.acm.org/citation.cfm?id=2737969>.
- [78] С. Дубаха, Дж. Кавазоса и Б. Франке. 2007. Быстрая оценка оптимизации компилятора с использованием производительности на основе функций кода-прогнозирование производительности. В *Труды 4-й Международной конференции по компьютерным рубежам*. 131-142. Извлечено из <http://dl.acm.org/citation.cfm?id=1242553>. [79] К. Дубах, Т. М. Джонс и Э. В. Бонилла. 2009. Портативный оптимизации компилятора через внедренные программы и микроархитектур с помощью машинного обучения. В *Материалы 42-го ежегодного Международного симпозиума IEEE / ACM по микроархитектуре*. ACM, 78-88. Извлечено из <http://dl.acm.org/citation.cfm?id=1669124>. [80] Крис Игл. 2011. *Книга iDA Pro: Неофициальное руководство по самому популярному в мире дизассемблеру*. Без крахмального пресса. [81] Хадди Эсмаилзаде, Эмили Блем, Рене Сент-Амант, Картикеян Шанкаралингам и Дуг Бургер. 2011. Темный кремний. *Материалы 38-го ежегодного Международного симпозиума по компьютерной архитектуре* и окончание многоядерного масштабирования. В *(ISCA'11)*. IEEE, 365-376.

- [82] Томас Л. Фальч и Энн К. Элстер. 2015. Автоматическая настройка на основе машинного обучения для повышения производительности opencl-портативность управления. В *Материалы семинара-практикума Международного симпозиума IEEE по параллельной и распределенной обработке данных (IPDPSW'15)*. IEEE, 1231-1240. [83] С. Фан, В. Сюй, Ю. Чен и Л. Эксаут. 2015. Практическая итеративная оптимизация для центра обработки данных. *ACM Trans. Arch. Код Optim.* 12, 2 (2015), 15. Извлечено из <http://dl.acm.org/citation.cfm?id=2739048>.
- [84] Паоло Фарабоски, Джеффри Браун, Джозеф А. Фишер, Джузеппе Десоли и Фред Хоумвуд. 2000. Лх: Технология платформа для настраиваемой встроенной обработки VLIW. В *Новостях компьютерной архитектуры ACM SIGARCH*, Том 28.ACM, 203-213.
- [85] Пол Фейтриер. 1988. Параметрическое целочисленное программирование. *РАЙРО Реч. Opération*. 22, 3 (1988), 243-268.
- [86] Жанна Ферранте, Карл Дж. Оттенштейн и Джо Д. Уоррен. 1987. График зависимости программы и его использование в оптимизации автоматизация. *ACM Trans. Программ. Язык. Система* 9, 3 (июль 1987), 319-349. doi : <http://dx.doi.org/10.1145/24039.24041>
- [87] Дж. А. Фишер, П. Фарабоски и К. Янг. 2009. Процессоры VLIW: когда-то blue sky, теперь стали обычным явлением. *Твердотельные процессоры IEEE Circ. Mag.* 1, 2 (2009), 10-17.
- [88] Джозеф А. Фишер. 1981. Сжатие микрокода. *Перевод IEEE. Вычисление*. 30, 7 (1981). [89] Джозеф А. Фишер, Паоло Фарабоски и Клифф Янг. 2004. *Встроенные вычисления: VLIW-подход к архитектуре, Компиляторы и инструменты*. Morgan Kaufmann.
- [90] Джозеф А. Фишер, Паоло Фарабоски и Клифф Янг. 2005. *Встроенные вычисления: VLIW-подход к архитектуре, Компиляторы и инструменты*. Elsevier.
- [91] Б. Франке, М. О'Бойл, Дж. Томсон и Г. Фурсин. 2005. Вероятностная оптимизация встроенных программ на уровне исходного кода-граммы. *Уведомления ACM SIGPLAN* (2005). Извлечено из <http://dl.acm.org/citation.cfm?id=1065922>.
- [92] Кристофер У. Фрейзер. 1999. Автоматический вывод моделей для статистического сжатия кода. *Примечания ACM SIGPLAN* 34, 5 (май 1999 г.), 242-246. <http://dx.doi.org/10.1145/301631.301672> doi :
- [93] Стефан М. Фрейденбергер и Джон К. Руттенберг. 1992. Посттапный порядок распределения регистров и схема инструкций-юлинг. В книге *Концепции, инструменты, техники генерации кода*. Спрингер, 146-170.
- [94] Джером Фридман, Тревор Хасты и Роберт Тишширани. 2001. *Элементы статистического обучения*. Том 1. Спрингер, Berlin.
- [95] Нир Фридман, Дэн Гейгер и Мойзес Гольдшмидт. 1997. Байесовские сетевые классификаторы. *Маш. Учиться* 29, 2-3 (1997), 131-163.
- [96] Г. Г. Фурсин. 2004. Итеративная компиляция и прогнозирование производительности для численных приложений. Извлечено из <https://www.era.lib.ed.ac.uk/handle/1842/565>.
- [97] Григорий Фурсин. 2010. Коллективный бенчмарк (cbench), набор программ с открытым исходным кодом и множеством наборов данных собран сообществом для обеспечения реалистичного бенчмаркинга и исследований по оптимизации программ и архитектуры. Извлечено из <http://ctuning.org/wiki/index.php/CTools:CBench>. [98] Г. Фурсин, Дж. Кавазос, М. О'Бойл и О. Темам. 2007. Промежуточные настройки: Создание условий для более реалистичного оценка итеративной оптимизации. *Материалы Международной конференции по высокопроизводительным встроенным Архитектуре и компиляторам*. 245-260. Извлечено из http://link.springer.com/chapter/10.1007/978-3-540-69338-3_3. [99] Г. Фурсин и А. Козн. 2007. Создание практического итеративного интерактивного компилятора. *Материалы семинара*. Извлечено от <https://www.researchgate.net/profile/Chuck>.
- [100] Г. Фурсин, А. Козн, М. О'Бойл и О. Темам. 2005. Практический метод быстрой оценки оптимизации программы. *Материалы Международной конференции по высокопроизводительным встроенным архитектурам и компиляторам*. 29-46. Извлечено из <http://link.springer.com/chapter/10.1007/11587514>. [101] Г. Фурсин, Ю. Кашников и А. В. Мемон. 2011. Milepost GCC: Самонастраивающийся компилятор с поддержкой машинного обучения. *Int. J. Параллельная программа*. 39, 3 (2011), 296-327. Извлечено из <http://link.springer.com/article/10.1007/s10766-010-0161-2>. [102] Григорий Фурсин, Антон Лохмотов и Эд Плауман. 2016. Коллективные знания: на пути к устойчивому развитию НИОКР. В *Материалах конференции и выставки "Проектирование, автоматизация и тестирование в Европе" (ДАТА ПРОВЕДЕНИЯ '16)*. IEEE, 864-869.
- [103] Григорий Фурсин, Антон Лохмотов, Дмитрий Савенко и Эбен Аптон. 2018. Рабочий процесс с коллективными знаниями для совместных исследований в области многоцелевой автоматической настройки и методов машинного обучения. *Препринт arXiv arXiv: 1801.08024* (2018).
- [104] Григорий Фурсин, Абдул Мемон, Кристоф Гионн и Антон Лохмотов. 2015. Коллективный разум, часть II: к программная инженерия с учетом производительности и затрат как естественная наука. *Препринт arXiv arXiv: 1506.06256* (2015).
- [105] Г. Фурсин, К. Миранда и О. Темам. 2008. MILEPOST GCC: компилятор исследований на основе машинного обучения. *Материалы саммита GCC*. Извлечено из <https://hal.inria.fr/inria-00294704/>.
- [106] Г. Г. Фурсин, М. Ф. П. О'Бойл и П. М. В. Кинненбург. 2002. Оценка итеративной компиляции. *Материалы Международного семинара по языкам и компиляторам для параллельных вычислений*. 362-376. Извлечено из <http://link.springer.com/chapter/10.1007/11596110>. [107] Григорий Фурсин и Оливье Темам. 2009. Коллективная оптимизация. В *Материалах Международной конференции по Высокопроизводительным встроенным архитектурам и компиляторам*. Springer, 34-49.

- [108] Г. Фурсин и О. Темам. 2010. Коллективная оптимизация: практический совместный подход. *ACM Trans. Архитектор. Код Optim.* 7, 4 (2010), 20. Извлечено из <http://dl.acm.org/citation.cfm?id=1880047>.
- [109] Davide Gadioli, Ricardo Nobre, Pedro Pinto, Emanuele Vitali, Amir H. Ashouri, Gianluca Palermo, Cristina Silvano, и Жоао Кардозу. 2018. SOCRATES - бесшовный онлайн-компилятор и платформа автоматической настройки среды выполнения для энергосберегающих приложений. В *Материалы конференции и выставки "Проектирование, автоматизация и тестирование в Европе" (DATA PROCESSING)*. 1143-1146. (118) *Материалы* 2018.8342183
- Гарсиарена и Роберто Сантана. 2016. Эволюционная оптимизация выбора флагов компилятора путем изучения и использование взаимодействий flags. В разделе *Материалы конференции 2016 года по генетическим и эволюционным вычислениям raiion (GECCO'16)*. ACM, Нью-Йорк, Нью-Йорк, 1159-1166. [111] Кириаос Георгиу, DOI : <http://dx.doi.org/10.1145/2908961.2931696>
- Крейг Блэкмор, Сэмюэл Ксавье-де Соуза и Керстин Эдер. 2018. Меньше значит больше: Эксплуатировать изменение стандартных уровней оптимизации компилятора для повышения производительности и энергопотребления. *Препринт arXiv arXiv: 1802.09845* (2018).
- [112] Чжан Сяоэнь Гун, Чжи Чен, Джастин Джозеф Сзадей, Дэвид К. Вонг, Зехра Сура, Нефтали Уоткинсон, Саид Малеки, David Padua, Alexandru Nicolau, Alexander V. Veldenbaum et al. 2018. Эмпирическое исследование влияния преобразований на уровне исходного кода на стабильность компилятора. В *Материалах семинара по компиляторам для параллельных вычислений (CPC'18)*. [113] Ричард Л. Горсач. 1988. Исследовательский факторный анализ. В *Справочнике по многомерной экспериментальной психологии*. Спрингер, 231-258.
- [114] Скотт Грауэр-Грей, Лифан Сюй, Роберт Сирлз, Судхи Аяласомаяджула и Джон Кавазос. 2012. Автоматическая настройка высокого-язык уровня, ориентированный на коды графических процессоров. В *Материалы конференции по инновационным параллельным вычислениям (InPar'12)*. IEEE, 1-10.
- [115] М. Холл, Д. Падуа и К. Пингали. 2009. Исследование компилятора: следующие 50 лет. *Commun. ACM* (2009). Извлечено из <http://dl.acm.org/citation.cfm?id=1461946>.
- [116] М. Ханеда. 2005. Оптимизация компилятора общего назначения. *Материалы 2-й конференции по вычислительной технике frontiers* (2005), 180-188. Извлечено из <http://dl.acm.org/citation.cfm?id=1062293>.
- [117] Тревор Хасты, Роберта Тибишрани и Джерома Фридмана. 2009. Обучение без учителя. В *Элементы статистики Обучение*. Спрингер, 485-585.
- [118] Джеффри Хайтауэр и Газзано Борриелло. 2001. Обзор и таксономия систем определения местоположения для повсеместных вычислений. *Вычисление по стандарту IEEE*. 34, 8 (2001), 57-66.
- [119] Джеффри Хинтон, Ли Дэн, Дон Ю, Джордж Э. Дэль, Абдель-Рахман Мохаммед, Навид Джайтли, Эндрю Сениор., Винсент Ванхоук, Патрик Нгуен, Тара Н. Сайнат и др. 2012. Глубокие нейронные сети для акустического моделирования в распознавании речи: общие взгляды четырех исследовательских групп. *Сигнальный процесс IEEE: Mag.* 29, 6 (2012), 82-97. [120] Торстен Хефнер и Роберто Белли. 2015. Научный бенчмаркинг параллельных вычислительных систем: двенадцать способов определить массы при представлении результатов производительности. В *Трудах Международной конференции по высокопроизводительным вычислениям, сетям, хранилищам и анализу*. ACM, 73. [121] Кеннет Хост и Ливен Экхаут. 2007. Микроархитектура-независимая характеристика рабочей нагрузки. *IEEE Micro* 27, 3 (2007), 63-72.
- [122] К. Хост и Л. Экхаут. 2008. Коул: Исследование уровня оптимизации компилятора. В *Материалы 6-го ежегодного конференции Международный симпозиум IEEE / ACM по генерации и оптимизации кода*. 165-174. Получено с сайта <http://dl.acm.org/citation.cfm?id=1356080>. [123] К. Хост, А. Джорж и Л. Экхаут. 2010. Автоматическая настройка компилятора точно в срок. *Материалы 8-й ежегодной Международный симпозиум IEEE / ACM по генерации и оптимизации кода* (2010), 62-72. Извлечено из <http://dl.acm.org/citation.cfm?id=1772965>.
- [124] Рональд А. Ховард. 1966. "Динамическое программирование". *Наука управления* 12, 5 (1966), 317-348.
- [125] Texas Instruments. 2012. Плата Pandaboard. *Плата разработчика SoC OMAP4430 2* (2012). [126] Кадзуаки Ишизаки, Акихиро Хаяси, Гита Кобленц и Вивек Саркар. 2015. Компиляция и оптимизация программ на Java 8 для выполнения на графическом процессоре. В *Материалах Международной конференции по параллельной архитектуре и компиляции (PACT'15)*. IEEE, 419-431.
- [127] Майкл Р. Янц и Прасад А. Кулкарни. 2013. Использование фазовых зависимостей для ускорения итеративного компилятора оптимизация поиска в порядке следования фаз. В *Материалах Международной конференции по компиляторам, архитектуре и синтезу для встраиваемых систем (KEIS'13)*. IEEE, 1-10. [128] Сверре Яри. 2002. *Методология использования счетчиков производительности Itanium 2 для анализа узких мест*. Технический отчет , HP Labs.
- [129] Брайан Джефф. 2012. Архитектура Big. LITTLE system от ARM: экономия энергии за счет гетерогенной многопроцессорности и миграции контекста задачи. В *Материалах 49-й ежегодной конференции по автоматизации проектирования*. ACM, 1143-1146.
- [130] Ричард Арнольд Джонсон и Дин У. Вичерн. 2002. *Прикладной многомерный статистический анализ*. Том 5. Прентис Холл, Аппер-Седл-Ривер, Нью-Джерси.

Материалы 6-го ежегодного конференции

- [131] Билл Джой, Гай Стил, Джеймс Гослинг и Гиллад Брача. 2000. *Спецификация языка Java (TM)*. Аддисон-Уэсли. [132] Agnieszka Kamiadska and Wacodzimierz Bielecki. 2016. Статистические модели для ускорения разработки программного обеспечения за счет средства итеративной компиляции. *Вычислений. Наука*. 17, 3 (2016), 407. Извлечено из <https://journals.agh.edu.pl/csci/article/просмотреть/1800>.
- [133] Кристоф Карцаклис, Оскар Эрнандес, Чунг-Син Су, Томас Ильше, Уэйн Жубер и Ричард Л. Грэм. 2012. HERCULES: система преобразования кода на основе шаблонов. В *Материалах 26-го Международного симпозиума IEEE по параллельной и распределенной обработке данных, семинаров и форума PhD (IPDPSW'12)*. IEEE, 574-583. [134] Кристоф Карцаклис, Ынджунг Парк и Джон Кавазос. 2014. HSL0T: циклические преобразования HERCULES с возможностью написания сценариев движок. В *Материалах 4-го Международного семинара по предметно-ориентированным языкам и высокоуровневым фреймворкам для высокопроизводительных вычислений*. IEEE Press, 31-41. [135] Василиос Келефурас. 2017. Методология, сокращающая пространство поиска шести преобразований компилятора путем обращения их объединяют как одну проблему и используют детали аппаратной архитектуры. *Вычисления* (2017), 1-24.
- [136] Уильям Киллиан, Ренато Микели, Ынджунг Парк, Марко Альварес и Джон Кавазос. 2014. Улучшение производительности. в ядрах с помощью эвристики автоматической векторизации компилятора. *PRACE-RLEU*. Извлечено из <http://www.prace-rleu/IMG/pdf/WP183.pdf>. [137] Кен Чжэ Ким. 2003. Прогнозирование финансовых временных рядов с использованием машин опорных векторов. *Нейрокомпьютинг* 55, 1 (2003), 307-319.
- [138] Антон Киндестам. 2017. Графические функции для оптимизации кода на основе машинного обучения. Магистр наук. Диссертация, KTH, urn: nbn:se:kth:diva-211444.
- [139] Тору Кисуки, П. Книненбург, М. О'Бойл и Х. Вайсхофф. 2000. Итеративная компиляция при оптимизации программы. В *Материалы конференции по компиляторам для параллельных компьютеров (CPC'00)*. Citeseer, 35-44.
- [140] Тору Кисуки, Питер М. В. Книненбург, Майк Ф. П. О'Бойл, Франсуа Боден и Гарри А. Г. Вайсхофф. 1999. Исследование-исследование гибкости при итеративной компиляции. В книге *Высокопроизводительные вычисления*. Спрингер, 121-132.
- [141] Тору Кисуки, Питер М. В. Книненбург, и Майкл Ф. П. О'Бойл. 2000. Комбинированный выбор размеров плиток и их расклатки факторы с использованием итеративной компиляции. В *Материалы Международной конференции 2000 года по параллельным архитектурам и методам компиляции (PACT'00)*. 237-248. [142] П. М. В. Книненбург, Т. Кисуки и doi: <http://dx.doi.org/10.1109/PACT.2000.888348>
- М. Ф. П. О'Бойл. 2003. Комбинированный выбор размеров плитки и коэффициентов расклатки с использованием итеративная компиляция. *J. Суперкомпьютер*. 24 (2003), 43-67.
- [143] А. Косеки, 1997. Метод оценки оптимального времени развертывания вложенных циклов. В *Труды 3-го В-международного симпозиума по параллельным архитектурам, алгоритмам и сетям (I-SPAN'97)*. 376-382. Извлечено из <http://ieeexplore.ieee.org/xpls/abs>. [144] Алекс Крижевски, Илья Суцкевер и Джеффри Э. Хинтон. 2012. Классификация Imagenet с глубокой сверточностью нейронные сети. В книге *Достижения в области нейронных систем обработки информации*. Curran Associates, inc., 1097-1105.
- [145] Дэвид К. Ку и Джованни Де Микели. 1992. Проектирование космических исследований. В *Высокоуровневый синтез ASIC в условиях временных ограничений и синхронизации*. Спрингер, 83-111.
- [146] П. Кулкарни, С. Хайнс и Дж. Хизер. 2004. Быстрый поиск эффективных последовательностей этапов оптимизации. *ACM SIGPLAN Примечания* 39, 6 (2004), 171-182. Извлечено из <http://dl.acm.org/citation.cfm?id=996863>.
- [147] Прасада А. Кулкарни, Майкла Р. Янца и Дэвида Б. Уолли. 2010. Повышение производительности и скорость поиска последовательности этапов оптимизации. В *Уведомления ACM Sigplan*, том 45. ACM, 95-104.
- [148] Прасад А. Кулкарни, Дэвид Б. Уолли и Гэри С. Тайсон. 2007. Оценка порядка поиска этапов эвристической оптимизации алгоритмы. В *Материалах Международного симпозиума по генерации и оптимизации кода (CGO'07)*. IEEE, 157-169. [149] Прасад А. Кулкарни doi: <http://dx.doi.org/10.1109/CGO.2007.9>
- Дэвид Б. Уолли, Гэри С. Тайсон и Джек У. Дэвидсон. 2006. Этап исчерпывающей оптимизации или-der space exploration. В *Материалах Международного симпозиума по генерации и оптимизации кода (CGO'06)*. IEEE, 13.
- [150] Прасад А. Кулкарни, Дэвид Б. Уолли, Гэри С. Тайсон и Джек У. Дэвидсон. 2009. Практическая исчерпывающая оптимизация исследование и оценка последовательности этапов. *ACM Trans. Archit. Код Optim.* 6, 1, статья 1 (апрель 2009 г.), doi: <http://dx.doi.org/10.1145/1509864.1509865>
- [151] С. Кулкарни и Дж. Кавазос. 2012. Решение проблемы упорядочения этапов оптимизации компилятора с помощью машинного обучения-инг. *Уведомления ACM SIGPLAN* (2012). Извлечено из <http://dl.acm.org/citation.cfm?id=2384628>.
- [152] С. Кулкарни и Дж. Кавазоса. 2013. Автоматическое построение встроенных эвристик с использованием машинного обучения. *Материалы Международного симпозиума IEEE/ACM по генерации и оптимизации кода (CGO'13)*. 1-12. Извлечено из <http://ieeexplore.ieee.org/xpls/abs>. [153] Т. Сатиш Кумар, С. Сактивел, С. Сушил Кумар и Н. Арун. 2014. Упорядочивание фаз компилятора и оптимизация MPI параметры времени выполнения с использованием эвристических алгоритмов на SMPS. *Int. J. Приложение. Eng. Res.* 9, 24 (2014), 30831-30851.
- [154] Крис Латтнер и Викрам Адве. 2004. LLVM: платформа компиляции для анализа и преобразования программ на протяжении всей жизни.-обсуждение. В *Материалы Международного симпозиума по генерации и оптимизации кода (CGO'04)*. IEEE, 75-86.

- [155] Х. Лезер, Э. Бонилла и М. О'Бойл. 2009. Автоматическая генерация функций для оптимизации на основе машинного обучения-компиляция. *Материалы Международного симпозиума по генерации и оптимизации кода (CGO'09)*. 81-91. Получено из <http://ieeexplore.ieee.org/xpls/abs>. [156] Брюс У. Леверетт, Родерик Джеффри Гальтон Кэттелл, Стивен О. Хоббс, Джозеф М. Ньюкомер, Эндрю Х. Райнер, Брюс Р. Шатц и Уильям А. Вульф. 1979. Обзор проекта "Компилятор качества производства". Университет Карнеги Меллон, факультет компьютерных наук. [157] Фэнцян Ли, Фейлонг Тан и Яо Шен. 2014. Интеллектуальный анализ функций для оптимизации компиляции на основе машинного обучения. *Материалы 8-й Международной конференции по инновационным мобильным и интернет-сервисам в повсеместных вычислениях (IMIS'14)*. 207-214. [158] Шен Ли, Юн Чжао. <http://link.springer.com/10.1109/IMIS.2014.26>
- Ан, Ричард Д. Стронг, Джей Б. Брокман, Дин М. Таллсен и Норман П. Джоупли. 2009. Маклат: Интегрированная платформа моделирования мощности, площади и синхронизации для многоядерных и многоядерных архитектур. В *Материалы 42-го ежегодного Международного симпозиума IEEE/ACM по микроархитектуре (MICRO'09)*. IEEE, 469-480.
- [159] Ю. Ли, Дж. Донгарра и С. Томов. 2009. Заметка об автоматической настройке GEMM для графических процессоров. *Материалы конференции по Вычислительной науке (ICCS'09)*. Извлечено из <http://link.springer.com/chapter/10.1007/978-3-642-01970-8>.
- [160] Хуэй Лю, Жунцай Чжао, Ци Ван и Инъян Ли. 2018. ALIC: Прогнозирование оптимизации компилятора с низкими накладными расходами Модель. Беспроводная персональная связь. Springer. doi: <http://dx.doi.org/10.1007/s11277-018-5479-x>
- [161] Винсент Лехнер. 1999. PolyLib: библиотека для манипулирования параметризованными многогранниками. Извлечено из Интернета по адресу <http://icps.u-strasbg.fr/PolyLib/>.
- [162] П. Лукусевски и Ф. Федики. 2009. Автоматическое снижение WCET с помощью машинного обучения на основе эвристики для Функция подстановкой. *Труды 3-го Совещания по статистике и машинного обучения, подходы к архитектуре и компиляции (смарт - '09)*. 1-15. Извлечено из <https://www.researchgate.net/profile/Peter.Luk>. [163] Пол Лукусевски, Саша Плазар, Хайко Фальк, Питер Марведель и Лотар Тиле. 2010. Многоцелевое исследование-оптимизация компилятора для систем реального времени. *Материалы 13-го Международного симпозиума IEEE по Объектно-компонентным/сервис-ориентированным распределенным вычислениям реального времени (ISORC'10)*, том 1. 115-122. <http://dx.doi.org/10.1109/ISORC.2010.15>
- [164] Дэвид Б. Лавман. 1977. Улучшение программы путем преобразования от источника к источнику. *Дж. АКМ* 24, 1 (1977), 121-145. [165] Чи Кен Лук, Роберт Кон, Роберт Мут, Хариш Патил, Артур Клаузер, Джефф Лоуни, Стивен Уоллес, Виджай Джанапа Редди и Ким Хейзелвуд. 2005. Вывод: создание настраиваемых инструментов анализа программ с помощью динамического инструментария. В *Материалы конференции ACM SIGPLAN 2005 года по разработке и внедрению языков программирования (PLDI'05)*. ACM, Нью-Йорк, Нью-Йорк, 190-200. doi: <http://dx.doi.org/10.1145/1065010.1065034>
- [166] Л. Ло, Ю. Чен, К. Бу, С. Лонг и Г. Фурсин. 2014. Поиск репрезентативных наборов оптимизаций для адаптивных многоверсионных приложений. *Препринт arXiv arXiv:1407.4075* (2014). Извлечено из <http://arxiv.org/abs/1407.4075>.
- [167] Скотт А. Мальке, Дэвид К. Лин, Уильям Й. Чен, Ричард Э. Хэнк и Роджер А. Брингманн. 1992. Эффективная поддержка compiler для предопределенного выполнения с использованием гиперблока. В *Информационном бюллетене ACM SIGMICRO*, Том 23. IEEE Computer Society Press, 45-54. [168] Джованни Мариани, Александр Бранкович, Джанлука Палермо, Джована Йович, Витторио Заккария и Кристина Сильвано. 2010. Методология исследования пространства проектирования на основе корреляции для многопроцессорных систем на кристалле. В *Материалы 47-й конференции ACM/IEEE по автоматизации проектирования (DAC'10)*. IEEE, 120-125.
- [169] Дж. Марс и Р. Хундт. 2009. Оптимизация на основе сценариев: платформа для статического обеспечения оперативной оптимизации. *Материалы 7-го ежегодного Международного симпозиума IEEE/ACM по генерации и оптимизации кода*. Извлечено из <http://dl.acm.org/citation.cfm?id=1545068>. [170] Л. Г. А. Мартинс и Р. Нобре. 2016. Выбор на основе кластеризации для исследования оптимизации компилятора последовательности. *ACM Trans. Architect. Code Optim.* 13, 1 (2016), 28. Извлечено из <http://dl.acm.org/citation.cfm?id=2883614>. [171] Луис Г. А. Мартинс, Рикардо Нобре, Александр К. Б. Дельбем, Эдуардо Маркес и Жуан М. П. Кардозу. 2014. Исследование-распределение последовательностей оптимизации компилятора с использованием выбора на основе кластеризации. В *Примечания ACM SIGPLAN*, Том 49. ACM, 63-72.
- [172] Эми Макговерн, Элиот Мосс и Эндрю Г. Барто. 1999. Планирование линейного кода с использованием обучения с подкреплением и внедрений. *Технический отчет № 99-23* (1999).
- [173] Эми Макговерн, Элиот Мосс и Эндрю Г. Барто. 2002. Создание базового планировщика инструкций по блокам с подкреплением-изучение улучшений и развертывание. *Mach. Учиться*. 49, 2-3 (2002), 141-160.
- [174] Абдул Вахид Мемон и Григорий Фурсин. 2013. Краудтонинг: систематизация автонастройки с использованием прогнозного моделирования и краудсорсинга. В *Материалах мини-симпозиума PARCO по автонастройке приложений для HPC (архитектур)*.
- [175] Р. Мичели, Г. Чиварио, А. Сикора и Э. Сезар. 2012. Автонастройка: основанный на плагилах подход к автоматической настройке параллельных приложений. *Материалы Международного семинара по прикладным параллельным вычислениям*. 328-342. Извлечено из <http://link.springer.com/chapter/10.1007/978-3-642-36803-5>.
- [176] MiniR 2011. Минимальное ИК-пространство. Извлечено из <http://www.assembla.com/wiki/show/minir-dev>.

DOI: <http://dx.doi.org/10.1007/978-3-642-36803-5>

- [177] Мехриар Мохри, Ашфин Ростамизаде., и Амит Талвалкар. 2012. *Основы машинного обучения*. Издательство Массачусетского технологического института. [178] А. Монсифро, Ф. Бодин и Р. Кинью. 2002. Подход машинного обучения к автоматическому созданию компилятора эвристика. (2002), 415-430. Извлечено из *Международной конференции по искусственному интеллекту: методология, системы и приложения*. <http://link.springer.com/chapter/10.1007/3-540-46148-5>.
- [179] Тьерри Моро, Антон Лохмотов и Григорий Фурсин. 2018. Представляем ReQuEST: открытую платформу для повторного-производимые и эффективные по качеству системы-турниры ML. *Korr*. abs/1801.06378. [arxiv: 1801.06378](http://arxiv.org/abs/1801.06378). Получено из <http://arxiv.org/abs/1801.06378>. [180] Элиот Мосс, Пол Утгофф, Джон Кавазос, Дойна Прекал, Д. Стефанович, Карла Бродли и Дэвид Шифф. 1998. Обучение планировать линейный код. *Дополнительная нейронная информация. Процесс. Система*. 10 (1998), 929-935. Извлечено из <http://books.nips.cc/документы/файлы/nips10/0929.pdf>.
- [181] Пасхалик Мнейс, Павлос Петуменос и Хью Лезер. 2015. Итеративная компиляция на мобильных устройствах. *Korr*. abs/1511.02603. Извлечено из <http://arxiv.org/abs/1511.02603>.
- [182] Филипа Дж. Муччи, Ширли Браун, Кристин Дин и Джорджа Хо. 1999. RAPI: портативный интерфейс для аппаратного обеспечения счетчики производительности. В разделе *Материалы конференции Группы пользователей HPCMP Министерства обороны*. 7-10.
- [183] М. Намолару, А. Козн и Г. Фурсин. 2010. Практические обобщения семантических свойств программ для машины обучения оптимизации. В *Труды Международной конференции по 2010 компиляторы, архитектуры и Синтезу для встраиваемых систем*. Получается от <http://dl.acm.org/citation.cfm?id=1878951>. [184] Рикардо Нобре, Рейс Луис и М. П. Кардосо Жоао. 2016. Упорядочивание фаз компилятора как ортогональный подход для снижения энергопотребления. В *Материалы 19-го семинара по компиляторам для параллельных вычислений (CPS'16)*. [185] Рикардо Нобре, Луис Г. А. Мартинс и Жоао М. П. Кардозу. 2015. Использование ранее полученного позиционирования оптимизаций для исследования фазового упорядочения. В *Материалы 18-го Международного семинара по программному обеспечению и компиляторам для встраиваемых систем. ACM*, 58-67. [186] Рикардо Нобре, Луис Г. А. Мартинс и Жуан М. П. Кардозу. 2016. Выбор этапов итеративного компилятора на основе графов и подход к упорядочению фаз. *Примечания ACM SIGPLAN*. 21: 30. <http://dx.doi.org/10.1145/2907950.2907959001>.
- [187] Ricardo Nobre, Luis Reis, and João M. P. Cardoso. 2018. Влияние порядка фаз компилятора при ориентации на графические процессоры. В *Материалы семинаров по параллельной обработке (Euro-Par'17)*, Дора Б. Херас и Люк Буже (Ред.). Спрингер Международное издательство, Cham, 427-438. [188] Хиротака Огава, Коуя Симура, Сатоши Мацуока, Фуюхико Маруяма, Юкихико Сохда и Ясунори Кимура. 2000. OpenJIT: открытый, отражающий JIT-компилятор для Java. В разделе *Материалы Европейской конференции по объектно-ориентированному программированию*. Спрингер, 362-387. [189] Уильям Ф. Огилви, Павлос Петуменос, Чжэн Ван и Хью Лезер. 2017. Минимизация затрат на итеративную компиляцию с активным обучением. В *Материалах Международного симпозиума IEEE / ACM по генерации кода и оптимизации (CGO'17)*. IEEE, 245-256. [190] Karl Joseph Ottenstein. 1978. Графики потоков данных как промежуточная программная форма. Докторская диссертация. Университет Пердью.
- [191] Дэвид А. Падуа и Майкл Дж. Вулф. 1986. Расширенная оптимизация компилятора для суперкомпьютеров. *Commun. ACM*. 29, 12 (1986), 1184-1201.
- [192] Г. Палермо, К. Сильвано, С. Вальсекки и В. Заккария. 2003. Методология системного уровня для быстрого решения множества задач проектное исследование космоса. В *Материалы 13-го симпозиума ACM по СБИС на Великих озерах. ACM*, 92-95.
- [193] Gianluca Palermo, Cristina Silvano, and Vittorio Zaccaria. 2005. Исследование пространства для многоцелевого проектирования встроенных систем ded-. *.I. Встроить. Вычислитель.* 1, 3 (2005), 305-316.
- [194] Gianluca Palermo, Cristina Silvano, and Vittorio Zaccaria. 2005. Исследование пространства для многоцелевого проектирования встроенных систем ded-. *.I. Встроить. Вычислитель.* 1, 3 (2005), 305-316.
- [195] Джеймс Паллистер, Саймон Дж. Холлис и Джереми Беннетт. 2013. Определение параметров компилятора для минимизации энергопотребления.-решение для встраиваемых платформ. *Вычисление. J.* 58, 1 (2013), 95-109.
- [196] З. Пан и Р. Эйгенманн. 2004. Оптимизация компилятора рейтингов для автоматической настройки производительности. *Материалы Конференции ACM / IEEE по суперкомпьютерам 2004 года*. 14. Извлечено из <http://dl.acm.org/citation.cfm?id=1049958>.
- [197] Желонг Пан и Рудольф Эйгенманн. 2006. Быстрое и эффективное взаимодействие оптимизаций компилятора для автоматического настройки производительности. В *Труды Международного симпозиума по генерации кода и оптимизации (гроз'06)*. IEEE, 12. [198] Э. Парк, Дж. Кавазос и М. А. Альварес. 2012. Использование описания программ на основе графиков для прогнозного моделирования. *Материалы Международного симпозиума по генерации и оптимизации кода*. 295-305. Получено с сайта <http://dl.acm.org/citation.cfm?id=2259042>. [199] Э. Парк, Дж. Кавазос и Л. Н. Пуше. 2013. Прогнозное моделирование в пространстве многогранной оптимизации. *Международная программа J. Параллельная*. 41, 5 (2013), 704-750. Извлечено из <http://link.springer.com/article/10.1007/s10766-013-0241-1>.

- [200] Парк Ынджунг, Кристоф Карцаклис и Джон Кавазос. 2014. HERCULES: сильные тенденции к повышению интеллектуальности прогнозного моделирование. *Материалы 43-й Международной конференции по параллельной обработке*. 172-181. doi : <http://dx.doi.org/10.1109/ICPP.2014.26>
- [201] Э. Парк, С. Купкарни и Дж. Кавазос. 2011. Оценка различных методов моделирования для итеративной компиляции. *Материалы 14-й Международной конференции по компиляторам, архитектурам и синтезу для встраиваемых систем (KEYS '11)*. ACM, 65-74. Извлечено из <http://dl.acm.org/citation.cfm?id=2038711>. [202] Пак Ын Чжон. 2015. Автоматический выбор оптимизаций компилятора с использованием характеристик программы и машинного обучения звание доктора философии. Диссертация, Университет Делавэра, США.
- [203] Дэвид А. Паттерсон и Джон Л. Хеннесси. 2013. *Организация и проектирование компьютера: аппаратно-программный интерфейс*. Ньюнес.
- [204] Жемчужина Иудей. 1985. *Байесовские сети: модель самоактивируемой памяти для доказательных рассуждений. Технический отчет Калифорнийского университета в Лос-Анджелесе номер отчета. CSD-850017; Материалы 7-й конференции Общества когнитивных наук, том 3*. 329-334.
- [205] Лесли Перес Касерес, Федерико Пагноцци, Альберто Францен и Томас Штютцле. 2018. Автоматическая настройка GCC с использованием iGase. В *Искусственная эволюция*, Эвелин Латтон, Пьеррик Легран, Пьер Парренд, Никола Монмарше и Марк Шенауэр (Ред.), Международное издательство Springer, Cham, 202-216. [206] Р. П. Дж. Пинкерс, П. М. В. Кининбург, М. Ханеда и Х. А. Г. Вайскофф. 2004. Статистический отбор вариантов компилятора-
. *Материалы ежегодного Международного симпозиума Компьютерного общества IEEE по моделированию, анализу и симуляции компьютерных и телекоммуникационных систем (MASCOTS '04)*. 494-501. doi : <http://dx.doi.org/10.1109/MASCOT.2004.1348395>
- [207] Л. Л. Поллок и М. Л. Соффа. 1990. Инкрементная глобальная оптимизация для более быстрых перекомпиляций. В *Трудах Международной конференции по компьютерным языкам*. 281-290.
- [208] Л. Н. Пуше и К. Бастул. 2007. Итеративная оптимизация в многогранной модели. Часть I: Одномерное время. *Материалы Международного симпозиума по генерации и оптимизации кода (CGO'07)*. 144-156. Извлечено из <http://ieeexplore.ieee.org/xpls/abs>. [209] Л. Н. Пуше, К. Бастул, А. Козн и Дж. Кавазос. 2008. Итеративная оптимизация в многогранной модели. Часть II: Многомерное время. *Уведомления ACM SIGPLAN*. Извлечено из <http://dl.acm.org/citation.cfm?id=1375594>.
- [210] Л. Н. Пуше и У. Бондзугула. 2010. В сочетании итеративный и на основе моделей оптимизации в автоматическом параллельном базисе lization. *Труды 2010 ACM/Международная конференция IEEE для высокопроизводительных вычислений, сетей, хранения и анализа*. 1-11. Извлечено из <http://dl.acm.org/citation.cfm?id=1884672>. [211] Louis-Noël Pouchet. 2012. Polybench: набор многогранных тестов. Извлечено из http://www.cs.ucla.edu/pouchet/программное_обеспечение/polybench/.
- [212] С. Пурини и Л. Джейн. 2013. Поиск хороших последовательностей оптимизации, охватывающих пространство программы. *ACM Trans. Arch. Comput. Optim. Optim. Код*. 9, 4 (2013), 56. Извлечено из <http://dl.acm.org/citation.cfm?id=2400715>.
- [213] Matthieu Stéphane Benoit Queva. 2007. Упорядочивание фаз в оптимизирующих компиляторах. Магистерская диссертация. Технический университет Дания, DTU, DK-2800 кг. Лундбю, Дания.
- [214] Франческо Риччи, Лиор Рокан и Бранча Шапира. 2011. Введение в справочник рекомендательных систем. В *Рекомендации-Справочник по рекомендательным системам*. Спрингер, 1-35. [215] Ранджит К. Рой. 2001. *Проектирование экспериментов с использованием подхода Тагучи: 16 шагов к улучшению продукта и процесса*. Wiley-Interscience.
- [216] Т. Рузира, М. Холл и П. Басу. 2017. Автоматизация автоматической настройки под управлением компилятора для поэтапного повышения производительности. В разделе *Материалы семинаров-практикумов Международного симпозиума IEEE по параллельной и распределенной обработке данных (IPDPSW'17)*. 1362-1371. doi : <http://dx.doi.org/10.1109/IPDPSW.2017.152>
- [217] Рикардо Набингер Санчес, Хосе Нельсон Амалар, Дуэйн Шафрон, Мариус Пирву и Марк Студли. 2011. Используя та-изучает стратегии компиляции для конкретных методов. В *Материалах 9-го ежегодного Международного симпозиума IEEE / ACM по генерации и оптимизации кода. posium*. 257-266. Извлечено из <http://dl.acm.org/citation.cfm?id=2190072>. [218] Вивек Саркар. 1997. Автоматический выбор преобразований высокого порядка в компиляторах IBM XL FORTRAN. *IBM J. Отв. разработчик*. 41, 3 (1997), 233-264.
- [219] В. Саркар. 2000. Оптимизированное развертывание вложенных циклов. *Материалы 14-й Международной конференции по Суперкомпьютерам-путин*. 153-166. Извлечено из <http://dl.acm.org/citation.cfm?id=335246>.
- [220] Robert R. Schaller. 1997. Закон Мура: прошлое, настоящее и будущее. *Спектр IEEE* 34, 6 (1997), 52-59.
- [221] Э. Шкуфза, Р. Шарма и А. Эйкен. 2014. Стохастическая оптимизация программ с плавающей запятой с настраиваемой точностью. *Уведомления ACM SIGPLAN*. Извлечено из <http://dl.acm.org/citation.cfm?id=2594302>.
- [222] Jürgen Schmidhuber. 2015. Глубокое обучение в нейронных сетях: обзор. *Нейронная сеть*. 61 (2015), 85-117.
- [223] Paul B. Schneck. 1973. Обзор методов оптимизации компилятора. В *Материалы ежегодной конференции ACM*. ACM, 106-113.
- [224] Bernhard Schölkopf. 2001. Трюк ядра с расстояниями. В *Достижения в области нейронных систем обработки информации*. Curran Associates, Inc., 301-307.

- [225] Cristina Silvano, Giovanni Agosta, Andrea Bartolini, Andrea Beccari, Luca Benini, Joao M. P. Cardoso, Carlo Kавациони, Радим Чмар, Ян Мартинович, Джанлука Палермо и др. 2015. ANTAREX-автонастройка и адаптивность подхода к энергоэффективным крупномасштабным HPC-системам. В *теоретическая наука и инженерия (CSE'15)*. IEEE, 343-346. *Материалы 18-й Международной конференции IEEE по Соп-*
- [226] Cristina Silvano, Giovanni Agosta, Andrea Bartolini, Andrea R Beccari, Luca Benini, João Bispo, Radim Cmar, João Депутат парламента Кардозо, Карло Кавациони, Ян Мартинович и др., 2016. Автотюнинга и адаптивность подход к энергетической эффек- тивнее экскалопсных вычислений систем: ANTAREXС подход. В *Материалы конференции и выставки "Проектирование, автоматизация и тестирование в Европе" (ДАТА ПРОВЕДЕНИЯ'16)*. IEEE, 708-713.
- [227] Cristina Silvano, Giovanni Agosta, Stefano Cherubin, Davide Gadioli, Gianluca Palermo, Andrea Bartolini, Luca Benini, Jan Martinovič, Martin Palkovič, Kateřina Slaninová et al. 2016. Подход ANTAREX к автоматической настройке и адаптивность для энергоэффективных HPC-систем. В *Материалах Международной конференции ACM по вычислительной технике Frontiers. ACM*, 288-293. [228] Кристина Сильвано, Андреа Бартолини, Андреа Беккари, Кандида Манельфи, Карло Кавациони, Давиде Гаддиоли, Эрвен Роу,
- Gianluca Palermo, Giovanni Agosta, Jan Martinovič et al. 2017. Инструмент ANTAREX flow для мониторинга и автоматической настройки энергоэффективных HPC-систем. В *Материалы Международной конференции по встраиваемым компьютерным системам: Архитектура, моделирование и имитационное моделирование (SAMOS'17)*.
- [229] Cristina Silvano, William Fornaciari, Gianluca Palermo, Vittorio Zaccaria, Fabrizio Castro, Marcos Martinez, Sara Бокио, Роберто Зафалон, Прабхат Авазаре, Герт Ванмербек и др. 2011. Multicube: пространство для многоцелевого проектирования исследование многоядерных архитектур. В *Материалах ежегодного симпозиума VLSI 2010*. Springer, 47-63. [230] Cristina Silvano, Gianluca Palermo, Giovanni Agosta, Amir H. Ashouri, Davide Gadioli, Stefano Cherubin, Emanuele Виталий, Лука Бенини, Андреа Бартолини, Даниэле Чезарини, Жоао Кардозу, Жоао Биспо, Педро Пинто, Риккардо Нобре, Эрвен Роу, Лоик Беснард, Имане Ласри, Нико Санна, Карло Кавациони, Радим Чмар, Ян Мартинович, Катержина Сланинова, Мартин Голасовски, Андреа Р. Беккари и Кандида Манельфи. 2018. Автонастройка и адаптивность в энергоэффективных системах HPC: инструментарий ANTAREX. В *Материалах конференции Computing Frontiers.ACM*. [231] Бернард В. Сильверман. 1986. *Оценка плотности для статистики и анализа данных*. Том 26. CRC press. [232] Ричард Столлман. 2001. Использование и перенос коллекции компиляторов GNU. В *Лаборатории искусственного интеллекта Массачусетского технологического института*. Citeseer.
- [233] Ричард М. Столлман и др., 2003. *Использование GCC: справочное руководство по сборке компиляторов GNU*. GNU Press. [234] Кеннет О. Стэнли. 2002. Эффективное обучение с подкреплением благодаря развивающимся топологиям нейронных сетей. В *Продолжить- материалы конференции по генетическим и эволюционным вычислениям (GECCO'02)*. Цитирую.
- [235] М. В. Стенсон. 2006. Автоматизация построения эвристики компилятора с использованием машинного обучения. Извлечено из http://groups.csail.mit.edu/commit/papers/2006/stephenson_phdthesis.pdf.
- [236] М. Стенсон и С. Амарасингхе. 2003. Мета-оптимизация: улучшение эвристики компилятора с помощью машинного обучения. 38, 5 (2003), 77-90. Извлечено из <http://dl.acm.org/citation.cfm?id=781141>.
- [237] М. Стенсона и С. Амарасингхе. 2005. Прогнозирование коэффициентов развертывания с использованием контролируемой классификации. В *Трудах Международного симпозиума по генерации и оптимизации кода*. doi : <http://dx.doi.org/10.1109/CGO.2005.29>
- [238] М. Стенсон и У. М. О'Рейли. 2003. Генетическое программирование в применении к эвристической оптимизации компилятора. *Продолжение- материалы Европейской конференции по генетическому программированию*. 238-253. Извлечено из <http://link.springer.com/chapter/10.1007/3-540-36599-0>.
- [239] Ralph E. Steuer. 1986. *Многокритериальная оптимизация: теория, вычисления и приложения*. Уайли. [240] К. Сток, Л. Н. Пуше и П. Садаппан. 2012. Использование машинного обучения для улучшения автоматической векторизации. *ACM Trans. Architect. Code Optim.* 8, 4 (2012), 50. Извлечено из <http://dl.acm.org/citation.cfm?id=2086729>.
- [241] Тосио Суганума, Такеши Огасавара, Микио Такеучи, Тосиаки Ясуэ, Мотохиро Кавахито, Кацуаки Ишизаки, Хидеаки Комацу и Тошио Накатани. 2000. Обзор компилятора IBM Java just-in-time compiler. *IBM Syst. J.* 39, 1 (2000), 175-193. [242] Cristian Țăruș, I-Hsin Chung, Jeffrey K. Hollingsworth и др., 2002. Активная гармония: на пути к автоматизированной производительности- настройка управления. В *Материалах конференции ACM / IEEE по суперкомпьютерам 2002 года*. IEEE Computer Society Press, 1-11. [243] Микеле Тартара и Стефано Креспи Реджицци. 2012. Параллельная итеративная компиляция: использование MapReduce для ускорения работы машинное обучение в компиляторах. В *Материалы 3-го международного семинара по MapReduce и его приложениям датируются*. ACM, 33-40. [244] Микеле Тартара и Стефано Креспи Реджицци. 2013. Непрерывное изучение эвристики компилятора. *ACM Trans. Архитектор. Код Optim.* 9, 4 (2013), 46.
- [245] Микеле Тартара и Стефано Креспи Реджицци. 2013. Непрерывное изучение эвристики компилятора. *ACM Trans. Archit. Code Optim.* 9, 4, статья 46 (январь 2013 г.). doi : <http://dx.doi.org/10.1145/2400682.2400705>
- [246] Джеральд Тессауро и Грегори Р. Гальперин. 1996. Улучшение политики в режиме онлайн с использованием поиска по методу Монте-Карло. В *Конференции по системам нейронной обработки информации (NIPS'96)*. Том 96, 1068-1074.

- [247] Брюс Томпсон. 2002. Статистический, практический и клинический: сколько видов значимости необходимо учитывать консультантам? *J. Counsel. Dev.* 80, 1 (2002), 64-71.
- [248] Дж. Томсон, М. О'Бойл, Г. Фурсин и Б. Франке. 2009. Сокращение времени обучения в однократном компиляторе на основе машинного обучения. *Материалы Международного семинара по языкам и компиляторам для параллельных вычислений*. 399-407. Получено из <http://link.springer.com/10.1007>. [249] А. Тивари, К. Чен и Дж. 2009. Масштабируемый фреймворк автоматической настройки для оптимизации компилятора. В *Профессиональный-материалы Международного симпозиума IEEE по параллельным и распределенным процессам (IPDPS'09)*. 1-12. Извлечено из <http://ieeexplore.ieee.org/xpls/abs>.
- [250] Георгиос Турнавитис, Чжэн Ван, Бьорн Франке и Майкл Ф. П. О'Бойл. 2009. На пути к целостному подходу к автоматическому распараллеливанию: интеграция обнаружения параллелизма на основе профилей и сопоставления на основе машинного обучения. *ACM Sigplan Отсутствует*. 44, 6 (2009), 177-187. doi : <http://dx.doi.org/10.1145/1543135.1542496>
- [251] С. Триантафиллис, М. Вачараджани, Н. Вачараджани и Д. И. Август. 2003. Оптимизация компилятора-исследование пространств. В *Материалах Международного симпозиума по генерации и оптимизации кода (CGO'03)*. IEEE, 204-215. doi : <http://dx.doi.org/10.1109/CGO.2003.1191546>
- [252] Эбен Алтон и Гарет Халфакри. 2014. *Руководство пользователя Raspberry Pi*. Джон Уайли и сыновья. [253] К. Васвани. 2007. Эмпирические модели, чувствительные к микроархитектуре, для оптимизации компиляторов. *Международный симпозиум о генерации и оптимизации кода (CGO'07)* (2007), 131-143. Извлечено из <http://ieeexplore.ieee.org/xpls/abs>.
- [254] Стивен Р. Бергдал. 1982. Фазовая связь и генерация констант в оптимизирующем компиляторе микрокода. *ACM SIG-Микроновика*. 13, 4 (1982), 125-133.
- [255] Ричард Вудук, Джеймс В. Деммел и Джефф А. Билмес. 2004. Статистические модели для эффективности на основе эмпирического поиска-настройка управления. *Int. J. Высокая производительность. Вычисление. Приложение*. 18, 1 (2004), 65-94.
- [256] Ричард В. Вудук. 2011. *Автонастройка* Спрингер, Бостон, Массачусетс, 102-105. http://dx.doi.org/10.1007/978-0-doi:367-99766-4_68
- [257] Дэвид У. Уолл. 1991. *Пределы параллелизма на уровне инструкций*. Том 19. ACM.
- [258] Вэй Ван, Джон Кавазос и Аллан Портерфилд. 2014. Автоматическая настройка энергии с использованием многогранного подхода. В *Профессиональный-итоги семинара по методам составления многогранников*.
- [259] З. Ван и М. Ф. П. О'Бойл. 2009. Сопоставление параллелизма с многоядерностью: подход, основанный на машинном обучении. *ACM Sigplan Hem*. 44, 4 (2009), 75-84. Извлечено из <http://dl.acm.org/citation.cfm?id=1504189>. [260] Тодд Уотерман. 2006. *Адаптивная Компиляция и встраивание*. Докторская диссертация, Университет Райса.
- [261] Дебора Уитфилд и Мэри Лу Соффа. 1991. Автоматическая генерация глобальных оптимизаторов. В *Примечания ACM SIGPLAN*. Том 26. ACM, 120-129.
- [262] Д. Уитфилд, М. Л. Соффа, Д. Уитфилд и М. Л. Соффа. 1990. Подход к упорядочиванию оптимизирующих преобразований. В разделе *Материалы 2-го симпозиума ACM SIGPLAN по принципам и практике параллельного программирования (PPOPP'90)*, Том 25. ACM Press, Нью-Йорк, Нью-Йорк, 137-146. doi : <http://dx.doi.org/10.1145/99163.99179>
- [263] Дебора Л. Уитфилд и Мэри Лу Соффа. 1997. Подход к изучению преобразований, улучшающих код. *ACM Trans. Програма. Lang. Система*. 19, 6 (ноябрь 1997), 1053-1084. doi : <http://dx.doi.org/10.1145/267959.267960>
- [264] Доран К. Уайлд. 1993. *Библиотека для выполнения многогранных операций*. Технический отчет № 785. IRISA.
- [265] Сэмюэл Уильямс, Эндрю Уотерман и Дэвид Паттерсон. 2009. Линия крыши: проникающая визуальная модель исполнения для многоядерных архитектур. *Commun. ACM*. 52, 4 (2009), 65-76.
- [266] Роберт П. Уилсон, Роберт С. Френч, Кристофер С. Уилсон, Саман П. Амарасингхе, Дженнифер М. Андерсон, Стив У. К. Цзян, Ши-Вэй Ляо, Чаувань Цзэн, Мэри У. Холл, Моника С. Лам и др. 1994. SUIF: инфраструктура для исследований по распараллеливанию и оптимизации компиляторов. *ACM Sigplan Отсутствует*. 29, 12 (1994), 31-37. [267] М. И. Волчок и Д. М. Унгар. 2000. Метод и устройство для повышения производительности компилятора во время последующего-постоянные компиляции исходной программы. *Патент США № 6,078,744*. Получено из <https://www.google.com/patents/US6078744>. [268] Стефан Вонг, Тийс Ван Ас и Джеффри Браун. 2008. ρ -VEK: Реконфигурируемый и расширяемый программный интерфейс VLIW-процессор. В *Трудах Международной конференции по технологии ICECE (FPT'08)*. IEEE, 369-372.
- [269] Уильям Аллан Вульф, Ричард К. Джонссон, Чарльз Б. Вайншток, Стивен О. Хоббс и Чарльз М. Гешке. 1975. *Разработка оптимизирующего компилятора*. Elsevier Science Inc.
- [270] Т. Юки, В. Басупалли, Г. Гупта, Г. Иоосс и Д. Ким. 2012. AlphaZ: система анализа, преобразования и программирования. генерация в полиздральной эквивалентной модели. Извлечено из <http://www.cs.colostate.edu/TechReports/Отчеты/2012/tr12-101.pdf>. [271] Т. Юки, Г. Гупта, Д. Г. Ким, Т. Патан и С. Раджападхье. 2012. AlphaZ: система для проектирования космических исследований в-многогранная модель, В *Материалы Международного семинара по языкам и компиляторам для параллельных вычислений*. 17-31. Извлечено из <http://people.rennes.inria.fr/Tomofumi.Yuki/документы/yuki-lcpc2012.pdf>.

[272] Vittorio Zaccaria, Gianluca Palermo, Fabrizio Castro, Cristina Silvano, and Giovanni Mariani. 2010. Мультикуб explorer: платформа с открытым исходным кодом для исследования пространства проектирования чиповых мультипроцессоров. В *Трудах 23-й Международной конференции по архитектуре вычислительных систем (ARCS'10)*. VDE, 1-7.

[273] Мин Чжао, Брюс Чайлдс, Мэри Лу Соффа, Мин Чжао, Брюс Чайлдс и Мэри Лу Соффа. 2003. Прогнозирование

влияния оптимизаций для встраиваемых систем. В *Материалах конференции ACM SIGPLAN 2003 года по языку, компилятору и инструменту для встраиваемых систем (LCTES'03)*, Том 38. ACM Press, Нью-Йорк, Нью-Йорк, 1.

DOI: <http://dx.doi.org/10.1145/780732.780734>

[274] Мин Чжао, Брюс Р. Чайлдс и Мэри Лу Соффа. 2005. Структура, основанная на моделях: подход к оптимизации, ориентированной на прибыль. В *Материалы Международного симпозиума по генерации и оптимизации кода*. IEEE, 317-327.

Получено в ноябре 2016 г.; пересмотрено в феврале 2018 г.; принято в марте 2018 г.